

HasteBoots: Proving FHE Bootstrapping in Seconds

Fengrun Liu^{*,1}, Haofei Liang², Tianyu Zhang², Yuncong Hu^{†,2}, Xiang Xie³, Haisheng Tan⁴, Yu Yu²

¹*Shanghai Qi Zhi Institute*, ²*Shanghai Jiao Tong University*, ³*Shanghai Boduo Technology Co., Ltd*,

⁴*University of Science and Technology of China*

ABSTRACT

Fully Homomorphic Encryption (FHE) enables computations on encrypted data, ensuring privacy for outsourced computation. However, verifying the integrity of FHE computations remains a significant challenge, especially for bootstrapping, the most computationally intensive operation in FHE. Prior approaches, including zkVM-based solutions and general-purpose SNARKs, suffer from inefficiencies, with proof generation times ranging from several hours to days. In this work, we propose HasteBoots, a succinct argument tailored for FHE operations. By designing customized polynomial interactive oracle proofs and optimized polynomial commitment schemes, HasteBoots achieves proof generation in a few seconds for FHE NAND with bootstrapping, significantly outperforming existing methods. Our approach demonstrates the potential for scalable and efficient verifiable FHE, paving the way for practical, privacy-preserving computations.

KEYWORDS

Succinct Arguments, Verifiable FHE

1 INTRODUCTION

Fully Homomorphic Encryption (FHE) [9, 10, 19, 21, 28, 29, 38] is a groundbreaking cryptographic technology that allows computations to be performed directly on encrypted data. This unique capability ensures that sensitive information remains protected throughout the computation process, making FHE especially valuable in scenarios that require privacy preservation, such as secure cloud computing and confidential data processing. However, achieving full homomorphism—where any arbitrary computation can be performed on encrypted data—relies on a key process known as bootstrapping [38]. Bootstrapping is essential in FHE because it periodically reduces accumulated noise in the ciphertext, enabling ongoing computation without data degradation, thereby maintaining both security and correctness.

In secure outsourced computing, Fully Homomorphic Encryption is often deployed to protect client data privacy, as it allows computations to be carried out on encrypted data. However, this setting introduces a unique challenge: while FHE ensures data confidentiality, it does not inherently guarantee that the computations performed on encrypted data are accurate or unaltered. For example, in outsourced machine learning scenarios—such as neural network models handled by cloud service providers—the client must have assurance that the inference or training computations

on their model are correct, despite not being able to access the raw (unencrypted) data or independently verify the results.

This is where Succinct Non-Interactive Arguments of Knowledge (SNARKs) [4, 7, 18, 20, 33, 41, 42, 56, 66] become essential. SNARKs provide a way to generate a compact, verifiable proof that the computations were executed correctly. This proof, which can be verified efficiently, allows the client to check the correctness of the computation without needing to trust the service provider. In decentralized systems, where the integrity of computations is crucial, FHE combined with SNARKs addresses the trust issue by providing verifiable results.

While some prior work [5, 30, 31, 35, 37, 39, 62] managed to prove basic FHE operations, such as addition and multiplication, these methods fail to support complex FHE operations, particularly bootstrapping; thus, these works cannot prove general FHE circuits. Bootstrapping is essential for achieving full homomorphism, as it resets accumulated noise in ciphertexts to allow ongoing computations. However, it is also the most computationally intensive operation in FHE, and how to prove bootstrapping efficiently remains a significant open problem in verifiable FHE.

One attempt is to leverage zero-knowledge virtual machines (zkVMs) [58, 59], which implement FHE algorithms directly in high-level language such as Rust, and then compile it to a proof. Although feasible, these zkVM-based solutions suffer from extremely low efficiency. Compiled code is not optimized for SNARKs, resulting in prohibitively long proof generation time, particularly for bootstrapping. Consequently, existing zkVMs like RISC0 [59] and SP1 [58] require approximately **three days** and **one day**, respectively, to generate a proof for a single bootstrapping.

A recent work [61] uses general-purpose SNARKs to directly verify FHE operations. [61] obtains performance improvements by eliminating the compilation from high-level languages. However, general-purpose SNARKs are not inherently optimized for FHE. For instance, FHE operations often involve complex relations such as Number Theoretic Transform (NTT), accumulator update, modulus switching, and conversion from a ring element to a polynomial of a multiplicative group. These specialized relations cannot be efficiently presented in the general circuits or custom lookup gates supported by existing general-purpose SNARKs, leading to efficiency loss. As a result, [61] needs about **half an hour** for proving the bootstrapping procedure, which limits the practical scalability of verifiable FHE.

Our goal is to construct a proof system specifically tailored for FHE with bootstrapping that can generate proofs in just a few seconds, making verifiable FHE computations viable for scalable, privacy-preserving computations without centralized trust.

* This work was completed during the author's internship at Shanghai Qi Zhi Institute.

† Corresponding author

1.1 Our Contributions

Our approach leverages the Polynomial Interactive Oracle Proofs (PIOPs) and Polynomial Commitment Schemes (PCS) paradigm to construct an efficient proof system specifically for FHE. Our contributions are as follows:

- **We propose a novel proof construction framework for FHE.** Our proof system targets FHE NAND with bootstrapping, as it is general enough to capture any FHE circuit. NAND is universal, and bootstrapping enables unbounded-depth homomorphic computations. We identify key atomic operations in the FHE NAND workflow as shown in Fig. 1 and build a new framework for proof construction as shown in Fig. 2. Based on this framework, we analyze FHE algorithms and identify FHEW/TFHE [21, 28] as the *proof-friendly FHE* scheme due to its simpler bootstrapping process, which is the most expensive operation in FHE. We further select FHE parameters that enable more efficient proof generation. Our framework provides a foundation for future work to optimize proofs for FHE where it suffices to improve the sub-protocols for each component of the framework.
- **We construct an efficient succinct argument HasteBoots for FHEW/TFHE.** We develop custom PIOPs optimized specifically for all internal FHE operations, leveraging their algebraic structure to significantly improve proof efficiency. This completeness enables an efficient PIOP for verifying the correctness of a functionally complete gate—FHE NAND with bootstrapping—which suffices to support arbitrary FHE computation. In particular, we achieve an optimal prover for the fast-NTT algorithm with bit-reversal, a widely used technique that may be of independent interest. We further propose an optimized PIOP for batched NTT operations on monomials, where the subclaim reduces to a single query on a sparse coefficient matrix, eliminating the need to prove the lift step typically required in FHE. Additionally, our modulus switching PIOP is novel, reducing the argument to simpler range checks without introducing additional noise. To balance proving efficiency with field-agnostic support across different settings, we select a PCS based on linear codes [41] and apply a packing mechanism to accelerate the opening on numerous small polynomials. As a result, we obtain a succinct argument HasteBoots for FHEW/TFHE with linear proving time.
- **We implement and evaluate HasteBoots.** Our evaluation demonstrates the efficiency of HasteBoots, with much faster proof generation than existing general-purpose proof systems. On Apple M4, HasteBoots can prove an FHE NAND operation with bootstrapping in 5 **seconds**, whereas the state-of-the-art [61] requires about half an hour.

1.2 Overview of Our Techniques

Workflow of FHE NAND. We first introduce the workflow of our FHE NAND procedure in Fig. 1. FHE NAND is a universal gate, including four main steps: LWE addition, batched NTT operations on monomials, accumulator update, and modulus switching.

The input consists of two LWE ciphertexts $ct_0 = (a_0, b_0) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ and $ct_1 = (a_1, b_1) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, encrypted under the secret key $s \in \mathbb{Z}_q^n$. The output is a LWE ciphertext $ct = (-a_0 - a_1, \frac{5q}{8} - b_0 - b_1) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, encrypted under the same secret key s , while the corresponding plaintext is the NAND result of the input plaintext.

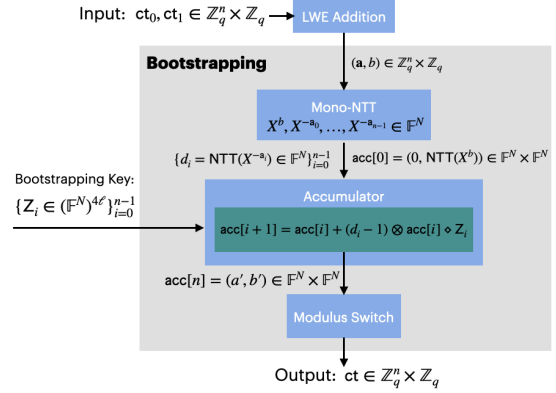


Figure 1: The computation flow of the NAND operation in FHE. The gray part represents the bootstrapping procedure, where the bootstrapping keys are public. We assume $q = 2N$ for simplicity. $\otimes$ and $\diamond$ are defined in Section 2.2.

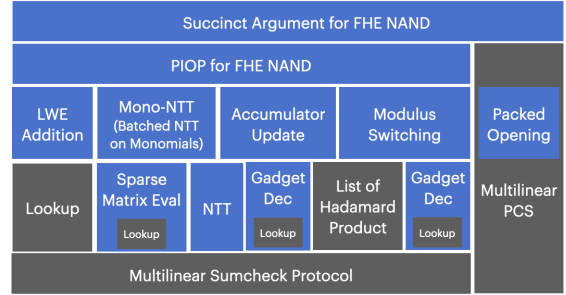


Figure 2: Protocol design overview of proof systems for FHE NAND, highlighting our newly designed protocols in blue and existing protocols in gray.

The FHE scheme first performs LWE addition on these ciphertexts to produce $(a, b) = (a_0 + a_1, b_0 + b_1)$, with increased noise. Here, the term $\frac{5q}{8}$ is omitted for simplicity. Bootstrapping aims to reduce this noise by homomorphically decrypting the ciphertext using a circular encryption of the secret key—i.e., the secret key encrypted under itself. This is denoted by Z_i , an RGSW ciphertext that encrypts the secret s_i . The decryption circuit for the LWE ciphertext (a, b) evaluates $\lfloor \frac{b - \langle a, s \rangle}{q/4} \rfloor$.

Our bootstrapping procedure uses an RLWE scheme [51] over a cyclotomic ring $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, with its quotient ring $\mathcal{R}_Q = \mathcal{R}/(QR) = \mathbb{F}[X]/(X^N + 1)$, where $\mathbb{F}$ is the finite field over the prime Q . The cyclotomic ring contains a cyclic subgroup $G = \{X, \dots, X^{2N}\}$, and a $2N$ -th root of unity ω such that $\omega^{2N} = 1$. To perform the homomorphic decryption in the RLWE scheme, we lift the decryption circuit into the exponent, computing $X^{b - \langle a, s \rangle}$. Assuming $q = 2N$ for simplicity, the lift step maps $b \in \mathbb{Z}_q$ to the cyclic group, resulting in a lifted polynomial $X^b \in G$. The accumulator is updated iteratively to obtain an RLWE ciphertext encrypting $X^{b - \langle a, s \rangle}$ with reduced noise. Finally, we extract an LWE ciphertext (a', b') under modulus Q from the accumulator's output and perform modulus switching to return it to modulus q . For each component $a'_i \in \mathbb{F}$, modulus switching computes $\lfloor \frac{a'_i \cdot q}{Q} \rfloor \in \mathbb{Z}_q$ by rounding to the nearest integer in $\mathbb{Z}_q$.

PIOP Design. We introduce our PIOP design as shown in Fig. 2 tailored for the FHE NAND workflow. We use multilinear polynomials to encode the FHE ciphertexts. By reducing all relations to a multilinear sumcheck protocol, we achieve an efficient linear-time prover. We select the FHE modulus Q to ensure that our PIOP can be defined on the finite field $\mathbb{F}$ over the prime Q , enabling native FHE arithmetic on $\mathbb{Z}_Q$. We further employ an extension field $\mathbb{EF}$ to improve the soundness.

LWE Addition. The core of proving addition between two **LWE** ciphertexts involves an addition modulo q . However, since our proof system uses $\mathbb{F}$ with a different modulus Q , we need to emulate this modulus operation within the field. Given the multilinear polynomials $\mathbf{a}, \mathbf{b}, \mathbf{c}$, the relation $\mathbf{c}(x) = \mathbf{a}(x) + \mathbf{b}(x) \bmod q$ holds for all $x \in \{0, 1\}^{\log n}$ if and only if there exists $\mathbf{k}$ such that $\mathbf{a}(x) + \mathbf{b}(x) = \mathbf{k}(x) \cdot q + \mathbf{c}(x)$ and $\mathbf{k}(x) \in \{0, 1\}$. This can be done efficiently using the multilinear sum-check protocol. We also need to prove that $\mathbf{c}(x)$ is within the correct range. We observe that since the modulus q in FHE is relatively small, it is efficient to employ a lookup argument with the table $(0, \dots, q-1)$.

Mono-NTT: Batched NTT on Monomials. We propose an efficient PIOP for batched NTT operations on monomials with an optimal prover. This protocol enables a direct reduction of the argument to a query on the output of the LWE addition, thereby eliminating the need to explicitly prove the *lifting* step—i.e., converting a ring element $b \in \mathbb{Z}_q$ into the exponent of a monomial X^b in the cyclic subgroup G (with exponent $b \cdot \frac{2N}{q}$, for simplicity, we assume $q = 2N$), as illustrated in Fig. 1. In FHE, the polynomial ring $\mathcal{R}_Q$ is defined over the irreducible polynomial $X^N + 1$, meaning that the lifted polynomial becomes X^b if $b < N$, and $-X^{b-N}$ otherwise. Establishing a direct relation between a ring element and its corresponding lifted polynomial is nontrivial, making it challenging to prove with existing techniques.

Crucially, we observe that the lifted polynomial is always a monomial—i.e., a polynomial with exactly one non-zero coefficient—whose exponent is determined by the input of the lift operation. Moreover, these monomials are only multiplied with other polynomials during the subsequent accumulator updates. Thus, instead of explicitly proving the lift operation, we prove batched NTT operations on M such monomials. Each monomial can be compactly represented by its coefficient and exponent: let $\mathbf{k} \in \mathbb{F}^M$ store the non-zero coefficients and $\mathbf{r} \in \mathbb{Z}_N^M$ store the corresponding exponents. This structure allows us to design an efficient batched NTT argument with optimal prover complexity $O(N + M)$, significantly improving over the naive $O(MN)$ approach.

Concretely, we randomize M NTT relations into a single NTT relation by leveraging the linearity of the NTT. Recall that the NTT maps a polynomial’s coefficient vector to its evaluations at N roots of unity. Given M such NTT relations (one per monomial), we construct a single randomized NTT relation via a random linear combination, using randomness $r' \in \mathbb{EF}^{\log M}$ (see Section 4.2 for details). This randomized relation can be computed in linear time, as each monomial has only one non-zero coefficient. It can also be verified in linear time via an NTT argument (further discussed in the next part), which reduces to querying the coefficient vector at a random point $r \in \mathbb{EF}^{\log M}$. Notably, this query corresponds

to an evaluation at the joint point (r, r') on a sparse $N \times M$ coefficient matrix, where each column is the full coefficient vector of one monomial. Prior works [20, 56, 57] observe that sparse matrices can be efficiently represented using three vectors—row, col and val—to locate the non-zero entries. In our case, this representation simplifies further: each column has exactly one non-zero value k_i at position r_i . Therefore, the evaluation proof at (r, r') reduces to queries on row = $\mathbf{r}$ and val = $\mathbf{k}$, both directly derived from the output of the LWE addition. The column vector col = $(0, \dots, M)$ is fixed and can be omitted.

Accumulator Update. The accumulator is initialized with an **RLWE** ciphertext $\text{acc}[0]$. Each update takes a circular encryption Z_i and the lifted polynomial (after NTT) $\mathbf{d}_i = \text{NTT}(X^{-a_i})$: $\text{acc}[i+1] = \text{acc}[i] + (\mathbf{d}_i - 1) \otimes \text{acc}[i] \diamond Z_i$. Both $\otimes$ and $\diamond$ (external product) are defined in Section 2.2. This process primarily involves Hadamard products, NTT operations, and gadget decompositions with respect to a tunable basis B . Hadamard products can be efficiently proven using sum-check protocols.

The gadget decomposition generalizes bit decomposition by breaking down the coefficients $\mathbf{f} \in \mathbb{F}^N$ of a polynomial into $\ell = \lceil \log_B Q \rceil$ small polynomials with coefficient vectors $\mathbf{f}_0, \dots, \mathbf{f}_{\ell-1} \in \mathbb{Z}_B^N$ such that $\mathbf{f} = \sum_{i=0}^{\ell-1} B^i \cdot \mathbf{f}_i$. This implies that we need to verify the relationship between $\mathbf{f}$ and $\mathbf{f}_i$ and also ensure that each $\mathbf{f}_i$ lies within a valid range. The former can be addressed using a polynomial identity test. For the latter, while in bit decomposition we can easily verify $(\mathbf{f}_i(x) - 1)\mathbf{f}_i(x) = 0$, gadget decomposition in FHE adopts a larger basis B to improve computational efficiency. However, this comes at the cost of proof efficiency, as the proving time grows quadratically with the degree. On the upside, a larger basis reduces the number of NTT operations and Hadamard products over the decomposed gadget polynomials, which can lead to a more efficient prover overall. To balance computational and proof efficiency, we choose a larger basis B and employ a lookup protocol to verify that $\mathbf{f}_i(x)$ values fall within the range $[0, B)$.

The NTT operation, defined over $\mathcal{R}_Q$ with a $2N$ -th root of unity ω , transforms a degree- N polynomial’s coefficient vector into an evaluation vector at N specially chosen points. This transformation can be efficiently computed in $O(N \log N)$ using the butterfly algorithm, known as fast-NTT [55]. More specifically, the NTT can be expressed using a Fourier-like matrix $F(\mathcal{Y}, \mathcal{X}) = \omega^{(2\mathcal{Y}+1) \cdot \mathcal{X}}$. Prior work [48] presents a linear-time argument for the FFT, based on the standard Fourier matrix defined by roots of unity. However, these techniques do not directly apply to the NTT, as its use of different roots of unity changes the structure of the Fourier matrix. Moreover, the fast-NTT algorithm employs a divide-and-conquer butterfly structure that outputs the evaluation vector in bit-reversed order—an aspect also exhibited in the FFT but overlooked in [48]. Consequently, their prover incurs an additional $O(N \log N)$ overhead to reorder the evaluation vector, degrading proof generation performance.

In this work, we propose a PIOP for the NTT operation that achieves optimal prover efficiency and, notably, explicitly accounts for bit-reversal. Specifically, we introduce a new Fourier-like matrix $F^R(\mathcal{Y}, \mathcal{X}) = \omega^{(2\mathcal{Y}^R+1) \cdot \mathcal{X}}$, which inherently encodes bit-reversal. Here, $\mathcal{Y}^R$ denotes the bit-reversal of $\mathcal{Y}$ (see Section 2.2 for its formal definition). This provides an exact expression for the NTT

relation between a coefficient vector $\mathbf{c}$ and its bit-reversed output $\mathbf{a}^R$, given by $\mathbf{a}^R = F^R \cdot \mathbf{c}$. Rewriting this relation in terms of multivariate polynomials, it suffices to verify the equality $\tilde{\mathbf{a}}^R(u) = \sum_{x \in \{0,1\}^{\log N}} \tilde{\mathbf{c}}(x) \tilde{F}^R(u, x)$ at a random point $u \in \mathbb{B}\mathbb{F}^{\log N}$ using the sum-check protocol. However, the prover requires quadratic time to compute the bookkeeping table $\tilde{F}^R(u, x)$ for all $x \in \{0,1\}^{\log N}$ to generate the sum-check messages with techniques from [66].

To achieve an optimal prover, we decompose the exponents into bitwise components and break the computation of $\tilde{F}^R(u, x)$ into logarithmic rounds. The key insight is that the term $\omega^{2^{\log N - i}} = \omega^{\frac{2N}{2^{i+1}}}$ corresponds to the 2^{i+1} -th roots of unity, denoted by $\omega_{2^{i+1}}$, meaning that $\omega_{2^{i+1}}^X$ takes only 2^{i+1} distinct values for all X . Hence, we decompose $\mathcal{X}$ and $\mathcal{Y}^R$ so that in the i -th round, the prover computes only $O(2^i)$ values. While [48] employs a similar idea, our decomposition method differs in structure (see Section 3.2 for details). This approach enables the prover to use a dynamic programming algorithm to compute the bookkeeping table in linear time, yielding a strictly linear-time prover that inherently supports bit-reversal.

Moreover, leveraging the linearity of the NTT operation, we propose a batching technique: instead of proving each NTT relation individually, we compute a single randomized NTT relation as a random linear combination of the M original NTT relations, and verify this combined relation using our NTT argument. As described in Mono-NTT, this technique yields an asymptotic improvement for the prover in the case of monomials, as M monomials contain only M non-zero coefficients. In addition, even when verifying NTT relation for general polynomials, this approach offers concrete performance gains in protocols over extension fields: it reduces $O(MN)$ extension field operations to $O(MN)$ multiplications between the base field and the extension field.

Modulus Switching. The final step in bootstrapping is modulus switching, which converts an $\mathbb{LWE}$ ciphertext under modulus Q to a new $\mathbb{LWE}$ ciphertext under a smaller modulus q . Specifically, this involves rounding $\frac{a \cdot q}{Q}$ to the nearest integer in $\mathbb{Z}_q$, as follows: $b = \lfloor \frac{a \cdot q}{Q} \rfloor \bmod q$. The intricate arithmetic of this operation makes proof generation challenging. A straightforward approach using lookup arguments to implement it as a custom gate results in an insufficient prover due to the large table size of Q . Instead, we exploit the inherent relationship between q and Q in FHE parameters to transform this complex operation into a highly structured mapping: one that assigns a range of values to the same integer in $\mathbb{Z}_q$ with the range itself parameterized by that output value. This mapping can then be verified using simpler range-check arguments.

Specifically, assuming that $2q \mid Q - 1$, we define $k = (Q - 1)/2q$, giving the relation $Q = 2kq + 1$. This structure allows us to partition the field into q ranges of length $2k$, along with a single special point. As shown in Fig. 3, for any $a \in [(2b - 1) \cdot k + 1, (2b + 1) \cdot k]$, it is mapped to $b \in \{1, \dots, q - 1\}$. The remaining uncovered range, $a \in [0, k] \cup (0, -k] \equiv Q$ is mapped to 0. Although this range consists of two separate ranges, they can be viewed as a single sequential range $[(2q - 1) \cdot k + 1, (2q + 1) \cdot k]$ of length $2k$ under modulus Q , along with a single point $a = k$. To ensure a consistent mapping, we introduce an auxiliary witness $b' \in \{1, \dots, q\}$ such that $b' \equiv b \bmod q$ to parameterize the range. As a result, this mapping can be expressed as a dichotomy: either $a = k$ and $b = 0$, or for any

$a \in [(2b' - 1) \cdot k + 1, (2b' + 1) \cdot k]$, we have a mapped to b where $b \in \mathbb{Z}_q$, and $b' \in \{1, \dots, q\}$ satisfies $b' \equiv b \bmod q$.

The structured mapping relation extracted from the modulus switching can be efficiently proven by standard techniques. Moreover, it faithfully emulates the modulus switching without introducing additional error into FHE ciphertexts, making it more flexible to different Q . In contrast, [61] introduces extra noise to FHE ciphertexts, limiting its applicability when Q is small.

Polynomial Commitments. Finally, to commit multilinear polynomials, we employ polynomial commitment schemes (PCS) compatible with FHE operations. We select Brakedown [40] due to its compatibility with the FHE field. Additionally, Brakedown is a linear-time protocol, which is crucial for our application.

We identify a challenge with using linear code-based PCS like Brakedown in FHE, where multiple small polynomials need to be opened. Although the total input size remains the same, Brakedown's performance drops significantly when handling multiple small polynomials compared to a single large polynomial, since the opening proofs rely on the relative code distance. Furthermore, Brakedown's opening proofs are not homomorphic, preventing batch openings. To solve this, we design a packing mechanism that combines M multilinear polynomials, each with $\log N$ variables, into a single $(\log M + \log N)$ -variate polynomial. Opening this packed polynomial at a random point reveals a random linear combination of the small polynomials evaluated at the corresponding random sub-point, reducing the overall cost.

2 PRELIMINARY

2.1 Notation

We denote the set $\{0, \dots, a - 1\}$ by $[a]$. Let $\mathbb{Z}_q$ be a ring over an integer q , and $\mathbb{F}_Q$ a field defined over the prime Q (omitted where clear). For a vector $\mathbf{a} \in \mathbb{F}^N$, the i -th entry is denoted by a_i . We use $\mathbf{a}[n_1..n_2]$ to denote the sub-vector of $\mathbf{a}$ starting at index n_1 and ending before index n_2 (i.e., excluding n_2). The vector $\mathbf{a}$ can be decomposed with respect to a basis $B \in \mathbb{Z}$ into $\ell = \lceil \log_B Q \rceil$ vectors $\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1}$ such that $\mathbf{a} = \sum_{i=0}^{\ell-1} B^i \cdot \mathbf{a}_i$ where $\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1} \in \mathbb{Z}_B^N$. This decomposition is referred to as a gadget decomposition and is denoted by $\text{Dec}(\mathbf{a}; B, \ell) = (\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1})$. For vectors $\mathbf{a}$ and $\mathbf{b}$, we define the Hadamard product as $\mathbf{c} = \mathbf{a} \circ \mathbf{b}$, where $c_i = a_i \cdot b_i$ for each i . The identity function $\mathbb{1}(x)$ returns 1 if the condition x is true, and 0 otherwise.

Bit-Reversed Representation. Throughout the paper, assume N is a power of two. Let $x \in \{0, 1\}^{\log N}$ denote binary variables and $X \in \mathbb{F}$ denotes a field variable. For $x = (x_0, \dots, x_{\log N - 1})$, define the canonical injection to- $\text{fld}(x) = \sum_{i=0}^{\log N - 1} 2^i \cdot x_i$, which maps $x \in \{0, 1\}^{\log N}$ to $\mathbb{F}$, and let to-bits(X) denote its inverse. For binary strings $x, y \in \{0, 1\}^{\log N}$, let $\mathcal{X}, \mathcal{Y} \in \mathbb{F}$ represent their corresponding field values for simplicity. For $\mathcal{X} \in \mathbb{F}$ corresponding to $x \in \{0, 1\}^{\log N}$, define its bit-reversed representation as $\mathcal{X}^R = \sum_{i=0}^{\log N - 1} 2^{\log N - 1 - i} \cdot x_i$. Additionally, we denote the bit-reversed order of the vector $\mathbf{a}$ by $\mathbf{a}^R = (a_{0^R}, \dots, a_{(N-1)^R}) \in \mathbb{F}^N$.

Multilinear Extensions of Vectors and Matrices. Let the identity function $\text{eq} : \{0, 1\}^{\log N} \times \{0, 1\}^{\log N} \rightarrow \{0, 1\}$ be defined as $\text{eq}(x, y) = 1$ if $x = y$, and 0 otherwise. The multilinear extension of

eq, denoted $\tilde{eq}(x, y)$, can be expressed as $\tilde{eq}(x, y) = \prod_{i=1}^{\log N} ((1 - x_i)(1 - y_i) + x_i y_i)$ where $x, y \in \mathbb{F}^{\log N}$. A vector $\mathbf{a} = (a_0, \dots, a_{N-1}) \in \mathbb{F}^N$ can be viewed as a multivariate polynomial $\mathbf{a} : \{0, 1\}^{\log N} \rightarrow \mathbb{F}$ such that for all $x \in \{0, 1\}^{\log N}$, we have $\mathbf{a}(x) = a_x$. The multilinear extension of $\tilde{a} : \mathbb{F}^{\log N} \rightarrow \mathbb{F}$ can be uniquely defined using the equality function as $\tilde{\mathbf{a}}(x_0, \dots, x_{\log N-1}) = \sum_{b \in \{0, 1\}^{\log N}} \tilde{eq}(x, b) \cdot \mathbf{a}(b)$ such that $\tilde{\mathbf{a}}(x) = \mathbf{a}(x)$ for all $x \in \{0, 1\}^{\log N}$. Similarly, the multilinear extension of an $N \times M$ matrix A , denoted by $\tilde{A}$, is the multilinear extension of the function $A : \{0, 1\}^{\log N + \log M} \rightarrow \mathbb{F}$.

Cyclotomic Ring. For N a power of two, the $2N$ -th cyclotomic ring is defined as $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, which has the $2N$ -th roots of unity, such that $\omega^{2N} = 1$. The quotient ring $\mathcal{R}_Q = \mathcal{R}/(Q\mathcal{R})$ consists of polynomials in $\mathcal{R}$ with the coefficients modulo Q . Any polynomial $c(X) = \sum_{i=0}^{N-1} c_i \cdot X^i \in \mathcal{R}_Q$ can be uniquely represented by the coefficient vector $\mathbf{c} = (c_0, \dots, c_{N-1}) \in \mathbb{F}^N$.

NTT/INTT. The Number Theoretic Transform (NTT) enables polynomial multiplication in $\mathcal{R}_Q$ via the fast-NTT algorithm [55]. NTT maps a polynomial $c(X) \in \mathcal{R}_Q$ from its coefficient vector $\mathbf{c}$ to an evaluation vector $\mathbf{a} = (a_0, \dots, a_{N-1}) \in \mathbb{F}^N$, where a_y evaluates at $X = \omega^{2y+1}$ for $y \in [N]$. Since the fast-NTT produces $\mathbf{a}$ in bit-reversed order, we define $\text{NTT}(\mathbf{c}) = \mathbf{a}^R$, and its inverse as $\text{INTT}(\mathbf{a}^R) = \mathbf{c}$.

Extension Field. Let $F(X) \in \mathbb{F}[X]$ be an irreducible polynomial of degree D . By the Chinese Remainder Theorem, we define the extension field $\mathbb{F}[X]/(F)$, denoted by $\mathbb{EF}$, which has size Q^D . Each field element in $\mathbb{EF}$ is represented as a polynomial of degree at most $D - 1$, and arithmetic within $\mathbb{EF}$ is performed using polynomial arithmetic modulo $F(X)$. Multilinear extensions of vectors and matrices can be similarly defined over $\mathbb{EF}$.

2.2 Fully Homomorphic Encryption

We focus on the FHEW-like fully homomorphic encryption systems with fast bootstrapping procedures. A Learning with Errors (LWE) [54] ciphertext is of the form $(\mathbf{a}, b) = (\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + mq/t + e \mod q)$ where $\mathbf{a} \leftarrow \mathbb{Z}_q^n$, $\mathbf{s} \in \{0, 1\}^n$ is the secret key, $e \leftarrow \chi_\sigma$ is sampled from some discrete gaussian distribution with small norm, and $m \in \mathbb{Z}_t$ is the message. To define the relation among various FHE ciphertexts, we use LWE to denote the vector space $\mathbb{Z}_q^{n+1}$, allowing us to represent a LWE ciphertext as a vector $\text{ct} = (\mathbf{a}, b) \in \text{LWE}$.

A Ring Learning with Errors (RLWE) [51] ciphertext is of the form $(\mathbf{a}, b) = (\mathbf{a}(X), \mathbf{a}(X) \cdot z(X) + \mathbf{m}(X) + \mathbf{e}(X))$ where $\mathbf{a}(X) \leftarrow \mathcal{R}_Q$, $z(X)$ is the secret polynomial, $\mathbf{e}(X) \leftarrow \chi_\sigma^N$ is sampled from a discrete gaussian distribution, and $\mathbf{m}(X)$ is the message polynomial. Based on the form of RLWE ciphertexts, we use RLWE to denote the space $\mathbb{F}^N \times \mathbb{F}^N$, so that an RLWE ciphertext is represented by $\mathbf{c} \in \text{RLWE}$, containing two vectors $\mathbf{c}.\mathbf{a} \in \mathbb{F}^N$ and $\mathbf{c}.\mathbf{b} \in \mathbb{F}^N$.

Let B be the basis for decomposition and $\ell = \lceil \log_B Q \rceil$. We define an RLWE' ciphertext as containing ℓ RLWE ciphertexts that encrypt the messages $\mathbf{m}, B\mathbf{m}, \dots, B^{\ell-1}\mathbf{m}$ under the same secret key. We define an RGSW ciphertext as containing two RLWE' ciphertexts that encrypt $-\mathbf{z}\mathbf{m}$ and $\mathbf{m}$, where $\mathbf{z}$ is the underlying secret polynomial. We use RGSW to denote the space $\text{RLWE}^\ell \times \text{RLWE}^\ell$, so we can represent an RGSW ciphertext $\mathbf{Z} \in \text{RGSW}$ as containing 4ℓ vectors $\mathbf{Z}.\mathbf{a}_0, \dots, \mathbf{Z}.\mathbf{a}_{2\ell-1}$ and $\mathbf{Z}.\mathbf{b}_0, \dots, \mathbf{Z}.\mathbf{b}_{2\ell-1}$. Let $\mathbf{Z}.\tilde{\mathbf{a}} = (\mathbf{Z}.\mathbf{a}_0, \dots, \mathbf{Z}.\mathbf{a}_{2\ell-1})$ and $\mathbf{Z}.\tilde{\mathbf{b}} = (\mathbf{Z}.\mathbf{b}_0, \dots, \mathbf{Z}.\mathbf{b}_{2\ell-1})$.

Next, we define the operation $\mathbb{F}^N \otimes \text{RLWE} \rightarrow \text{RLWE}$: for a vector $\mathbf{d} \in \mathbb{F}^N$ and $\mathbf{c} \in \text{RLWE}$, we define $\mathbf{d} \otimes \text{RLWE} = (\text{INTT}(\mathbf{d} \circ \mathbf{c}.\mathbf{a}), \text{INTT}(\mathbf{d} \circ \mathbf{c}.\mathbf{b})) \in \text{RLWE}$. We also define the operation $\text{RLWE} \otimes \text{RGSW} \rightarrow \text{RLWE}$ as follows: for $\mathbf{c}' \in \text{RLWE}$ and $\mathbf{Z} \in \text{RGSW}$, we first decompose $\mathbf{c}'.\mathbf{a}$ and $\mathbf{c}'.\mathbf{b}$ into 2ℓ vectors, denoted by $\text{bits}[0..2\ell] = (\mathbf{c}'.\mathbf{a}_0, \dots, \mathbf{c}'.\mathbf{a}_{\ell-1}, \mathbf{c}'.\mathbf{b}_0, \dots, \mathbf{c}'.\mathbf{b}_{\ell-1})$. We then perform NTT on each vector, corresponding to a decomposed polynomial, to obtain $\text{Nbts}[0..2\ell]$, where each $\text{Nbts}[i] = \text{NTT}(\text{bits}[i])$. Finally, we compute

$$\left(\sum_{i=0}^{2\ell-1} \text{Nbts}[i] \circ \mathbf{Z}.\mathbf{a}_i, \sum_{i=0}^{2\ell-1} \text{Nbts}[i] \circ \mathbf{Z}.\mathbf{b}_i \right) \in \text{RLWE}$$

This operation is often referred to as the external product.

We provide a brief overview of the entire FHE NAND with bootstrapping in Algorithm 3 and Appendix A and refer the reader to [53] for detailed explanations.

2.3 Useful PIOP

Sum-check Protocol. Let $\mathcal{G}$ be a μ -variate polynomial defined over a field $\mathbb{F}$. The goal of the sum-check protocol is to allow a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of a sum: $T = \sum_{x \in \{0, 1\}^\mu} \mathcal{G}(x_1, x_2, \dots, x_\mu)$. Lund et al. [50] proposed a sum-check protocol that allows the verifier to efficiently verify the claimed sum. The protocol results in linear proving time and logarithmic proof size by delegating the computation to the prover. See Appendix B.1 for the formal definition of the sum-check protocol.

Hadamard Protocol. We define a Hadamard protocol as a PIOP for the relation $\mathcal{R}_o$ (see Definition 1), which involves a sum of M Hadamard products. The protocol construction (see Appendix B.2) follows directly from the sum-check protocol, achieving perfect completeness and soundness error $O(\log N)/|\mathbb{EF}|$. The prover runs in $O(MN)$ time, and the verifier runs in $O(\log N + M)$ where M corresponds to the query complexity for evaluating $\mathcal{G}$. The communication cost of the protocol is $O(\log N)$ when the degree of $\mathcal{G}$ is constant, which holds in our constructions.

DEF 1. The Hadamard relation $\mathcal{R}_o$ is the set of tuples $(i, \mathbf{x}, \mathbf{w}) =$

$$(\perp, (\mathbb{F}, \mathbb{EF}, N, M), (\{\mathbf{a}_0, \dots, \mathbf{a}_{M-1}\}, \{\mathbf{b}_0, \dots, \mathbf{b}_{M-1}\}, \mathbf{c}))$$

such that for all i , $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c} \in \mathbb{F}^N$ and $\sum_{i=0}^{M-1} \mathbf{a}_i \circ \mathbf{b}_i = \mathbf{c}$.

Lookups Based on the Logarithmic Derivative. Lookup arguments are used to prove that a sequence of values belongs to a prescribed table. Haböck proposed a lookup argument based on the logarithmic derivative in [43]. According to Lemma 5 in [43], there exists a PIOP protocol (described in Appendix B.3) for the batched lookup relation $\mathcal{R}_{\text{lookup}}$ (see Definition 2) that achieves perfect completeness and soundness error $O(MN)/|\mathbb{EF}|$, with communication cost of $O(\log N + M)$ field elements. The prover runs in $O(MN)$, and the verifier in $O(\log N + M)$. For convenience, we also define a variant—the indexed lookup relation $\mathcal{R}_{\text{id-lookup}}$ —which extends $\mathcal{R}_{\text{lookup}}$ by attaching an index to each lookup query.

DEF 2. The batched lookup relation $\mathcal{R}_{\text{lookup}}$ is the set of tuples

$$(i, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, t, M), (f_0, \dots, f_{M-1}))$$

where $t, f_0, \dots, f_{M-1} \in \mathbb{F}^N$ and $\bigcup_{i=0}^{M-1} \{f_i(x) \mid x \in \{0, 1\}^{\log N}\} \subseteq \{t(x) \mid x \in \{0, 1\}^{\log N}\}$ (as sets).

DEF 3. The indexed lookup relation $\mathcal{R}_{\text{id-lookup}}$ consists of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, M, \mathbf{t}), (\text{id}, \mathbf{f}))$$

where $\mathbf{t}, \mathbf{f} \in \mathbb{F}^N$, $\text{id} \in \mathbb{Z}_N^N$ and $\{(\text{id}(x), \mathbf{f}(x)) \mid x \in \{0, 1\}^{\log N}\} \subseteq \{(\text{to-fld}(x), \mathbf{t}(x)) \mid x \in \{0, 1\}^{\log N}\}$ as sets.

3 PIOP FOR BUILDING BLOCKS

3.1 Simplified Sparse Matrix Evaluation

[56, 57] proposed a standard commitment scheme for sparse multilinear polynomials with optimal prover costs, in which the sparse matrices are represented by dense polynomials. The evaluation proof in this case is essentially a specialized PIOP with dense polynomials being committed. In this section, we consider a specialized sparse $N \times M$ matrix C in which each column contains exactly one non-zero entry. Building on Claim 1 from [56], we formalize the evaluation of this special matrix structure in Lemma 3.1 and define the corresponding relation, $\mathcal{R}_{\text{sparse}}$, in Definition 4. Our construction, presented in Appendix C.1, simplifies the original approach by replacing the memory-checking technique with an indexed lookup argument, since the table size is small in our setting.

LEMMA 3.1. Given a $\log N + \log M$ -variate multilinear polynomial $\tilde{C}$ defined over a sparse $N \times M$ matrix C , in which each column has only one non-zero entry, there exist two $(\log M)$ -variate multilinear polynomials row and col such that the following holds for all $r_x \in \mathbb{EF}^{\log N}$, $r_y \in \mathbb{EF}^{\log M}$:

$$\tilde{C}(r_x, r_y) = \sum_{k \in \{0, 1\}^{\log M}} \text{val}(k) \cdot \tilde{e}q(\text{to-bits}(\text{row}(k)), r_x) \cdot \tilde{e}q(k, r_y)$$

PROOF. The proof follows Claim 1 in [57], with $\text{col}(k) = \text{to-fld}(k)$ for all $k \in \{0, 1\}^{\log M}$. $\square$

DEF 4. The sparse matrix evaluation relation $\mathcal{R}_{\text{sparse}}$ is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, M, e, r_x, r_y), (\text{val}, \text{row}))$$

where $r_x \in \mathbb{EF}^{\log N}$, $r_y \in \mathbb{EF}^{\log M}$, $e \in \mathbb{EF}$, $\text{val} \in \mathbb{F}^M$, $\text{row} \in \mathbb{Z}_M^M$, and $\sum_{k \in \{0, 1\}^{\log M}} \text{val}(k) \cdot \tilde{e}q(\text{to-bits}(\text{row}(k)), r_x) \cdot \tilde{e}q(k, r_y) = e$.

THEOREM 3.2. For every finite field $\mathbb{F}$ and its extension field $\mathbb{EF}$, and for all positive integers N, M , there exists a PIOP for the relation $\mathcal{R}_{\text{sparse}}$ that achieves perfect completeness and soundness error $O(M)/|\mathbb{EF}|$, with communication cost of $O(\log M)$. The prover runs in $O(M)$ time, while the verifier runs in $O(\log M)$ time.

We prove it with Construction 3 in Appendix C.1

3.2 NTT/INTT

In this section, we propose a specialized PIOP for the NTT/INTT operation that inherently supports bit-reversal of the evaluation vector, achieving a strictly linear-time prover for NTT without additional overhead to reorder the outputs. Additionally, we propose an optimization for proving batched NTT operations by leveraging the linearity of the NTT operation.

In our work, the NTT operation transforms a polynomial's coefficient vector $\mathbf{c} \in \mathbb{F}^N$ into an evaluation vector $\mathbf{a} \in \mathbb{F}^N$, evaluated at $X = \omega^{2^{\mathcal{Y}+1}}$ for $\mathcal{Y} \in [N]$, with the output vector arranged in a bit-reversed order, i.e., $\mathbf{a}^R$. Specifically, the $\mathcal{Y}$ -th entry

of $\mathbf{a}^R$ corresponds to the evaluation at $X = \omega^{2^{\mathcal{Y}+1}}$. Hence, the NTT operation can be expressed as a matrix-vector multiplication $\mathbf{a}^R = F^R \cdot \mathbf{c}$, where F^R is a Fourier-like matrix defined by $F^R(\mathcal{Y}, \mathcal{X}) = \omega^{(2^{\mathcal{Y}+1}) \cdot \mathcal{X}}$. To prove this matrix-vector multiplication, we first rewrite the polynomial evaluation equation as a multivariate polynomial expression:

$$\tilde{\mathbf{a}}^R(y) = \sum_{x \in \{0, 1\}^{\log N}} \tilde{\mathbf{c}}(x) \tilde{F}^R(y, x) \quad (1)$$

for $y \in \{0, 1\}^{\log N}$. Given the evaluation $\tilde{\mathbf{a}}^R(u)$ at a random point $u \in \mathbb{EF}^{\log N}$, the proof reduces to a sum-check protocol applied to Eq. (1). Thaler [60] introduces a dynamic programming algorithm that initializes the values of $\tilde{\mathbf{c}}(x)$ for all $x \in \{0, 1\}^{\log N}$ in linear time. This precomputed table, referred to as the bookkeeping table in [66], enables the prover to generate sum-check messages efficiently. Prior work [48] proposes a linear-time algorithm for computing the bookkeeping table for $\tilde{F}(u, x)$ when proving FFT, where the Fourier matrix is defined by $F(\mathcal{Y}, \mathcal{X}) = \omega^{\mathcal{Y}\mathcal{X}}$ for $\mathcal{Y} \in [M]$, $\mathcal{X} \in [N]$. However, this approach cannot be directly applied to compute the bookkeeping table for $\tilde{F}^R(u, x)$ in the case of fast NTT, due to the distinct structure of the matrix F^R .

Instead, we adopt a similar strategy by decomposing the exponents of the root of unity ω . The term $\omega^{2^{\log N - i}} = \omega^{\frac{2^N}{2^{i+1}}}$ corresponds to the 2^{i+1} -th roots of unity, denoted as $\omega_{2^{i+1}}$, where $\omega_{2^{i+1}}^X$ takes on 2^{i+1} distinct values for all $X \in [N]$. We express $\tilde{F}^R(u, x)$ as follows:

$$\tilde{F}^R(u, x) = \sum_{y \in \{0, 1\}^{\log N}} \tilde{e}q(u, y) \tilde{F}^R(y, x) \quad (2)$$

$$= \sum_{y \in \{0, 1\}^{\log N}} \tilde{e}q(u, y) \omega^{(2^{\mathcal{Y}+1}) \cdot \mathcal{X}} \quad (3)$$

$$= \omega^{\mathcal{X}} \cdot \sum_{y \in \{0, 1\}^{\log N}} \tilde{e}q(u, y) \omega^{\mathcal{X} \cdot 2^{\mathcal{Y}}} \quad (4)$$

$$= \prod_{i=0}^{\log N - 1} (1 - u_i + u_i \cdot \omega_{2^{i+1}}^{\mathcal{X}}) \cdot \omega^{2^i \cdot x_i} \quad (5)$$

In Eq. (4), we decompose the bits of the first $\mathcal{X}$ and of $\mathcal{Y}^R$, while keeping the second $\mathcal{X}$ intact. This enables us to express each evaluation into a grand product form. The full derivation is provided in Appendix E.2. Note that Eq. (5) includes an extra term, $\omega^{2^i \cdot x_i}$, in the grand product compared to Eq. (8) in [48], due to the different structure of the new Fourier-like matrix used in our NTT proof.

Using the property that $\omega_{2^{i+1}}^{\mathcal{X}}$ takes on $2^i + 1$ distinct values, we divide the computation of $\tilde{F}^R(u, x)$ into $\log N$ rounds and compute the bookkeeping table via dynamic programming, as described in the Algorithm 1. In particular, Step 4 of Algorithm 1 computes the most significant bit (MSB) of j , which indicates x_i for the i -th round, where i ranges from 0 to $\log N - 1$.

Reducing the verifier time. In the final round of the sum-check protocol, the verifier must evaluate $\tilde{F}^R(u, \cdot)$ at a random point $r \xleftarrow{\$} \mathbb{EF}^{\log N}$. Directly evaluating it via Algorithm 1 would require linear time for the verifier, so we delegate the computation to the prover using a sequence of sum-check protocols, reducing the verifier's runtime to $O(\log^2 N)$. Specifically, we adopt a delegation protocol inspired by [48], leveraging the fact that the computation

Algorithm 1 $A_{FR} \leftarrow \text{Initialize}(\omega, \mu, N)$

Input: The degree N and the $2N$ -th root of unity ω .

Input: The random point $u \in \mathbb{F}^{\log N}$.

Output: A_{FR} storing $\tilde{F}^R(u, x)$ for all $x \in \{0, 1\}^{\log N}$

```

1:  $A_{FR}[0] = 1$ 
2: for  $i = 0, \dots, \log N - 1$  do
3:   for  $j = 2^{i+1} - 1, \dots, 0$  do
4:      $b = (j \gg i) \& 1$  // the MSB of  $j$ 
5:      $A_{FR}[j] = A_{FR}[j \% 2^i] \cdot ((1 - u_i) + u_i \cdot \omega_{2^{i+1}}^j) \cdot \omega^{2^i \cdot b}$ 
6: return  $A_{FR}$ 

```

in Algorithm 1 naturally forms a structured layered circuit. The delegation protocol consists of $\log N$ sum-check protocols, each corresponding to an iteration of the algorithm. In round i , verifying the correctness of array A_{FR} of size 2^{i+1} reduces to checking an evaluation of the prior round's array of size 2^i at a random point. This recursive process terminates at the base case, where $A_{FR}[0] = 1$, as initialized in Step 1. See more details in Appendix E.1.

By delegating the computation of $\tilde{F}^R(u, v)$, the verifier only needs to evaluate $\tilde{c}(\cdot)$ at the random point r . Consequently, we view the full PIOP (including the delegation protocol) for the NTT relation $\mathcal{R}_{NTT}$ as a mechanism that reduces the original check $\sum_{x \in \{0, 1\}^{\log N}} \tilde{c}(x) \cdot \tilde{F}^R(u, x) \stackrel{?}{=} e$ to the simpler subclaim of $\tilde{c}(r) \stackrel{?}{=} d$.

DEF 5. The NTT relation $\mathcal{R}_{NTT}$ is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, F^R, N, u, e), (c))$$

where $u \in \mathbb{E}\mathbb{F}^{\log N}$, $c \in \mathbb{E}\mathbb{F}^N$, and $e = \sum_{x \in \{0, 1\}^{\log N}} c(x) \cdot \tilde{F}^R(u, x)$.

THEOREM 3.3. For every finite field $\mathbb{F}$ and its extension field $\mathbb{E}\mathbb{F}$, and for all positive integers N , there exists a PIOP for relation $\mathcal{R}_{NTT}$ that achieves perfect completeness and soundness error $O(\log^2 N)/|\mathbb{E}\mathbb{F}|$, with communication cost of $O(\log^2 N)$. The prover runs in $O(N)$ time, and the verifier in $O(\log^2 N)$ time.

PROOF. Theorem 3.3 follows from a construction that combines a sum-check protocol—with a bookkeeping table computed by Algorithm 1—with a delegation protocol described in Appendix E.1. $\square$

Batched NTT Operations. Leveraging the linearity of the NTT operation, we can efficiently prove multiple NTT instances by reducing them to a single randomized NTT check. This technique not only optimizes the protocol for batched NTT operations on monomials (see Section 4.2), but also yields concrete performance improvements in practice.

Suppose the prover wishes to convince the verifier of the correctness of M NTT instances, each simply expressed as (u, e_i, c_i) , where c_i is the i -th coefficient vector and $e_i = \sum_{x \in \{0, 1\}^{\log N}} c_i(x) \cdot \tilde{F}^R(u, x)$. All instances share the same evaluation point $u \in \mathbb{E}\mathbb{F}^{\log N}$. To batch these, we sample random scalars $\rho_0, \dots, \rho_{M-1} \xleftarrow{\$} \mathbb{E}\mathbb{F}$ and form a single randomized instance (u, e', c') , where $c' = \sum_{i=0}^{M-1} \rho_i \cdot c_i$ and $e' = \sum_{i=0}^{M-1} \rho_i \cdot e_i$. Correctness follows from the linearity of NTT operation: $e' = \sum_{i=0}^{M-1} \rho_i \cdot \sum_x c_i(x) \cdot \tilde{F}^R(u, x) = \sum_x (\sum_{i=0}^{M-1} \rho_i \cdot c_i(x)) \cdot \tilde{F}^R(u, x) = \sum_x c'(x) \cdot \tilde{F}^R(u, x)$. The randomization introduces a soundness error of $1/|\mathbb{E}\mathbb{F}|$, which is negligible. The prover computes the randomized coefficient vector and then proves a single

NTT instance; the former step dominates the runtime for general (dense) coefficient vectors. However, if the coefficient vectors are sparse with only $O(M)$ non-zero entries, the prover time reduces to $O(M + N)$. This batching technique is especially beneficial when the protocol is executed over an extension field while the original NTT instances are defined over the base field. The prover can reduce $O(MN)$ extension field operations to $O(MN)$ base-to-extension field multiplications, significantly improving concrete performance.

Alternatively, instead of sampling M random scalars, we may view $(e_0, \dots, e_{M-1})$ as defining a multilinear extension and check the equality by evaluating it at a random point $r' \in \mathbb{E}\mathbb{F}^{\log M}$. The same derivation holds with scalars $\rho_i = \tilde{e}q(\text{to-bits}(i), r')$. This variant incurs a soundness error of $\log M/|\mathbb{E}\mathbb{F}|$ by Schwartz-Zippel lemma and requires only $\log M$ random elements. Used in our PIOP for batched NTT operations on monomials, this approach facilitates reducing the argument to evaluating a sparse $N \times M$ coefficient matrix at a joint random point (r, r') , where $r \in \mathbb{E}\mathbb{F}^{\log N}$ is the query point derived from the subclaim of the NTT check on (u, e', c') .

3.3 Gadget Decomposition

In this section, we present a PIOP for the operation $\text{Dec}(\mathbf{a}; B, \ell)$ that decomposes a vector $\mathbf{a} \in \mathbb{F}^N$, with basis B , into ℓ gadgets $\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1} \in \mathbb{Z}_B^N$ such that $\mathbf{a} = \sum_{i=0}^{\ell-1} B^i \cdot \mathbf{a}_i$, where $\ell = \lceil \log_B Q \rceil$. Compared to bit decomposition, FHE typically adopts a relatively larger basis B to improve computational efficiency. A larger basis also simplifies the arithmetic circuit in the accumulator, reducing the number of NTT operations and Hadamard products over decomposed gadgets. To ensure the efficiency both in FHE computation and proof generation, we utilize the PIOP for the batched lookup relation $\mathcal{R}_{\text{lookup}}$ (defined in Section 2.3) to check that the range of decomposed gadgets is within the table $\mathbf{t} = (0, \dots, B - 1)$.

DEF 6. The decomposition relation $\mathcal{R}_{\text{dec}}$ is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, B, \ell), (\mathbf{a}, (\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1})))$$

where $\ell = \lceil \log_B Q \rceil$, $\mathbf{a} \in \mathbb{F}^N$, $\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1} \in \mathbb{Z}_B^N$, $\mathbf{a} = \sum_{i=0}^{\ell-1} B^i \cdot \mathbf{a}_i$.

THEOREM 3.4. For every finite field $\mathbb{F}$ and its extension field $\mathbb{E}\mathbb{F}$, and for all positive integers N, B , there exists a PIOP for the relation $\mathcal{R}_{\text{dec}}$ that achieves perfect completeness and soundness error $O(\ell N)/|\mathbb{E}\mathbb{F}|$, with communication cost $\log N + \ell$. The prover runs $O(\ell N)$ time, and the verifier runs in $O(\log N + \ell)$ time.

We prove it with Construction 4 in Appendix C.2.

Extension to Range Check. We generally use the lookup argument to prove $\mathbf{a}$ lies in a range $[0, L]$, using a table $\mathbf{t} = (0, \dots, L - 1)$, when L is relatively small. However, when $L = O(Q)$ is large—approaching the size of $\mathbb{F}$ —we instead employ gadget decomposition to check the range relation $\mathcal{R}_{\text{range}}$ (Definition 7), since the prover time for lookup arguments increases linearly with the table size. When the range size L is not an exact power of B , let $\ell = \lceil \log_B L \rceil$. The prover computes the gadget decompositions of $\mathbf{a}$ and $\mathbf{a} + (B^\ell - L)$ and uses two invocations of the decomposition protocol to ensure that the intersection of the two verified ranges is exactly $[0, L]$. The detailed PIOP for $\mathcal{R}_{\text{range}}$ is omitted here.

DEF 7. The range check relation $\mathcal{R}_{\text{range}}$ is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, B, \ell, L), \mathbf{a})$$

where $\mathbf{a} \in \mathbb{Z}_L^N$ and $\ell = \lceil \log_B L \rceil$.

4 PIOP FOR FHE OPERATIONS

In this section, we introduce our PIOPs for the operations involved in the FHE NAND circuit, as illustrated in Fig. 1.

4.1 LWE Addition

The core idea behind proving LWE addition is to emulate a straightforward addition modulo q within our proof systems, which operates over modulus Q . Given $a, b \in \mathbb{Z}_q$, the result $c = a + b \bmod q$ holds if and only if there exists $k \in \{0, 1\}$ such that $c \in \mathbb{Z}_q$ and $a + b = k \cdot q + c$. To ensure that c lies within the correct range, we employ a lookup argument using the table $\mathbf{t} = (0, \dots, q-1)$. This is feasible in our design since the modulus q used in the LWE scheme is relatively small.

DEF 8. The LWE addition relation $\mathcal{R}_{\text{add}}$ is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (q, N, \mathbf{a}, \mathbf{b}), \mathbf{c})$$

where $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_q^N$ and $\mathbf{a} + \mathbf{b} = \mathbf{c} \pmod{q}$.

THEOREM 4.1. For every finite field $\mathbb{F}$ and extension field $\mathbb{EF}$, and for all positive integers q, N , there exists a PIOP for the relation $\mathcal{R}_{\text{add}}$ with perfect completeness and soundness error $O(N)/|\mathbb{EF}|$. The prover runs in $O(N)$ time, and the verifier in $O(\log N)$ time.

We prove it with Construction 5 in Appendix C.3.

4.2 Mono-NTT: Batched NTT on Monomials

In this section, we present a PIOP for batched NTT operations on monomials. This construction offers two key advantages. First, since each resulting polynomial after the lift step is a monomial with a single nonzero coefficient and is only multiplied with other polynomials during accumulator updates, we can leverage the sparsity of their coefficient vectors to efficiently prove batched NTT, as described in Section 3.2. Second, the argument for batched NTT on monomials reduces to querying the monomials' coefficients and exponents, which are directly derived from the output of the LWE addition. This allows us to eliminate the need to explicitly prove the lift step—i.e., converting a ring element into the exponent of a polynomial as illustrated in Fig. 1.

As previously shown in Fig. 1, the output of the LWE addition lies in the ring $\mathbb{Z}_q$. To homomorphically evaluate the decryption circuit in the RLWE scheme, we first lift each $a \in \mathbb{Z}_q$ into the exponent of a monomial $X^b \in G$, where $G = \{X, \dots, X^{2N}\}$ is a cyclic subgroup of the polynomial ring $\mathcal{R}_Q = \mathbb{F}/(X^N + 1)$. Since $q \mid 2N$, this is done by computing $b = a \cdot \frac{2N}{q} \in \mathbb{Z}_{2N}$. To ensure correct representation within $\mathcal{R}_Q$, we reduce X^b modulo $X^N + 1$ when $b \geq N$. The result of this modular reduction is X^b if $b < N$ and $-X^{b-N}$ otherwise.

Although the resulting polynomials are monomials with coefficients of 1 or -1 , we consider a general set of M monomials of the form $\{k_0 \cdot X^{r_0}, k_1 \cdot X^{r_1}, \dots, k_{M-1} \cdot X^{r_{M-1}}\}$ in $\mathcal{R}_Q$, where $r_i \in [N]$ and $k_i \in \mathbb{F}$. These monomials can be compactly represented by two vectors: $\mathbf{k} = (k_0, \dots, k_{M-1})$, which stores the coefficients, and $\mathbf{r} = (r_0, \dots, r_{M-1})$, which stores the exponents. However, to check the NTT computation for each monomial, we must represent it using its full coefficient vector, which also serves as the standard witness in the NTT relation $\mathcal{R}_{\text{NTT}}$. A simple observation is that the i -th monomial corresponds to a coefficient vector $\mathbf{c}_i$ with a

single nonzero entry k_i at position r_i . Stacking these vectors forms a sparse coefficient matrix $C^{N \times M} = (\mathbf{c}_0, \dots, \mathbf{c}_{M-1})$, where each column contains exactly one nonzero element k_i , located at position r_i . This matrix corresponds precisely to the one considered in Section 3.1, with the sparse representation given by row $= \mathbf{r}$ and val $= \mathbf{k}$. As a result, evaluating $C^{N \times M}$ at a random point reduces to subclaims that query only $\mathbf{r}$ and $\mathbf{k}$.

The remaining step is to efficiently reduce the batched NTT argument to a single query on the sparse coefficient matrix $C^{N \times M}$. As discussed in Section 3.2, a batch of NTT instances can be proven via a single randomized NTT instance. Specifically, consider M NTT operations on the coefficient vectors of monomials, denoted by $\mathbf{c}_0, \dots, \mathbf{c}_{M-1} \in \mathbb{F}^N$. Each NTT instance can be uniquely labeled by the tuple $(u, \mathbf{e}_Y, \mathbf{c}_Y)$ for $Y \in [M]$, according to Definition 5. In Section 3.2, we introduced two approaches for randomizing batched NTT instances. Here, we adopt the second method, which treats the batch as a multilinear extension and checks the equality by evaluating it at a random point $\mathbf{r}' \xleftarrow{\$} \mathbb{EF}^{\log M}$. More concretely, the randomized instance is defined as $(u, \mathbf{e}', \mathbf{c}')$, where $\mathbf{e}' = \sum_{y \in \{0,1\}^{\log M}} \tilde{e}q(y, \mathbf{r}') \cdot \mathbf{e}_Y$ and $\mathbf{c}' = \sum_{y \in \{0,1\}^{\log M}} \tilde{e}q(y, \mathbf{r}') \cdot \mathbf{c}_Y \in \mathbb{F}^N$. Since each $\mathbf{c}_Y$ has only one nonzero entry, and all scalars $\tilde{e}q(y, \mathbf{r}')$ for $y \in \{0,1\}^{\log M}$ can be jointly precomputed in linear time, the randomized vector $\mathbf{c}'$ can be efficiently computed in $O(M)$. Then the prover and the verifier run the PIOP from Theorem 3.3 on this randomized NTT instance. This yields a subclaim of an evaluation query on $\tilde{\mathbf{c}}'(r)$ for a random $r \xleftarrow{\$} \mathbb{EF}^{\log N}$. Notably, this query corresponds exactly to an evaluation of the sparse coefficient matrix C at the joint point $(r, \mathbf{r}')$:

$$\tilde{\mathbf{c}}'(r) = \sum_{y \in \{0,1\}^{\log M}} \tilde{e}q(y, \mathbf{r}') \tilde{\mathbf{c}}_Y(r) \quad (6)$$

$$= \sum_{y \in \{0,1\}^{\log M}} \tilde{e}q(y, \mathbf{r}') \sum_{x \in \{0,1\}^{\log N}} \tilde{e}q(x, r) \mathbf{c}_Y(x) \quad (7)$$

$$= \sum_{y \in \{0,1\}^{\log M}} \sum_{x \in \{0,1\}^{\log N}} \tilde{e}q(y, \mathbf{r}') \tilde{e}q(x, r) C(\mathbf{x}, \mathbf{y}) \quad (8)$$

$$= \tilde{C}(r, \mathbf{r}') \quad (9)$$

This means the query can be answered directly using the PIOP for sparse matrix evaluation described in Section 3.1. We now formally define our Mono-NTT relation $\mathcal{R}_{\text{Mono-NTT}}$, which captures M NTT operations performed on monomials.

DEF 9. The Mono-NTT relation $\mathcal{R}_{\text{Mono-NTT}}$ is the set of tuples

$$(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, M, u), (\mathbf{k}, \mathbf{r}, \mathbf{e}))$$

such that for each $i \in [M]$, the tuple $(\perp, (\mathbb{F}, \mathbb{EF}, F^R, N, u, e_i), (\mathbf{c}_i)) \in \mathcal{R}_{\text{NTT}}$, where $\mathbf{c}_i \in \mathbb{F}^N$ has exactly one nonzero entry $k_i \in \mathbb{F}$ at position $r_i \in \mathbb{Z}_N$.

THEOREM 4.2. For every finite field $\mathbb{F}$ and its extension field $\mathbb{EF}$, and for all positive integers N, M , there exists a PIOP for $\mathcal{R}_{\text{Mono-NTT}}$ that achieves perfect completeness and soundness error $O(\log^2 N + M)/|\mathbb{EF}|$, with communication cost $O(\log^2 N + \log M)$. The prover runs in $O(M + N)$ time, and the verifier runs in $O(\log^2 N + \log M)$.

We prove it with the Construction 6 in Appendix C.4.

Algorithm 2 $\text{AccUpdate}(\text{acc}, \mathbf{d}, Z, B, \ell)$

Input: $\text{acc} \in \text{RLWE}$, $\mathbf{d} \in \mathbb{F}^N$, $Z \in \text{RGSW}$

Input: $B \in \mathbb{Z}$ and $\ell = \lceil \log_B Q \rceil$

Output: $\text{acc}' \in \text{RLWE}$ and trace

```

1:  $\text{mult} = (\mathbf{d} \circ \text{acc}.\mathbf{a}, \mathbf{d} \circ \text{acc}.\mathbf{b}) \in \text{RLWE}$ 
2:  $\text{Imult} = (\text{INTT}(\text{mult}.\mathbf{a}), \text{INTT}(\text{mult}.\mathbf{b})) \in \text{RLWE}$ 
3:  $\text{bits}[0..\ell] = \text{Dec}(\text{Imult}.\mathbf{a}; B, \ell)$ 
4:  $\text{bits}[\ell..2\ell] = \text{Dec}(\text{Imult}.\mathbf{b}; B, \ell)$ 
5: for  $i = 0$  to  $2\ell$  do
6:    $\text{Nbits}[i] = \text{NTT}(\text{bits}[i])$ 
7: end for
8:  $\text{prod} = \left( \sum_{i=0}^{2\ell-1} \text{Nbits}[i] \circ Z.\mathbf{a}_i, \sum_{i=0}^{2\ell-1} \text{Nbits}[i] \circ Z.\mathbf{b}_i \right)$ 
9:  $\text{acc}' = \text{acc} + \text{prod} \in \text{RLWE}$ 
10: return  $\text{acc}'$  and  $(\text{mult}, \text{Imult}, \text{bits}, \text{Nbits}, \text{prod})$ 
```

4.3 Accumulator Update

In this section, we propose a PIOP for proving the correctness of an accumulator update of the form: $\text{acc}' = \text{acc} + \mathbf{d} \circ \text{acc} \circ Z$, which updates the accumulator state from acc to acc' . As described in Section 2.2, $\text{acc} \in \text{RLWE}$ (and similarly for acc') consists of two vectors $\text{acc}.\mathbf{a}$ and $\text{acc}.\mathbf{b}$ in $\mathbb{F}^N$. Each update consumes a vector $\mathbf{d} \in \mathbb{F}^N$ and an element $Z \in \text{RGSW}$, where Z contains 4ℓ vectors denoted by $Z.\mathbf{a}_0, \dots, Z.\mathbf{a}_{2\ell-1}, Z.\mathbf{b}_0, \dots, Z.\mathbf{b}_{2\ell-1} \in \mathbb{F}^N$. Here $\ell = \lceil \log_B Q \rceil$ denotes the number of gadgets in the gadget decomposition. This update is formalized in Algorithm 2, where the internal steps of the computation are recorded in trace.

In Algorithm 2, each accumulator update consists of $2\ell + 2$ NTT/INTT operations, two gadget decompositions, and $4\ell + 2$ Hadamard products. The basis B is a tunable parameter: choosing a larger basis B reduces both the number of NTT operations and Hadamard products, potentially resulting in a more efficient prover. The PIOP for proving the accumulator update relation $\mathcal{R}_{\text{acc}}$, defined below, follows naturally from the aforementioned PIOPs.

DEF 10. The accumulator update relation $\mathcal{R}_{\text{acc}}$ is the set of tuples $(i, \mathbf{x}, \mathbf{w}) = (Z, (\mathbb{F}, \mathbb{EF}, F^R, N, B, \ell), (\text{acc}, \mathbf{d}, \text{acc}', \text{trace}))$ where $\ell = \lceil \log_B Q \rceil$, $Z \in \text{RGSW}$, $\text{acc}, \text{acc}' \in \text{RLWE}$, $\mathbf{d} \in \mathbb{F}^N$, and $(\text{acc}', \text{traces}) = \text{AccUpdate}(\text{acc}, \mathbf{d}, Z, B, \ell)$. The components of trace are defined in Algorithm 2.

THEOREM 4.3. For every finite field $\mathbb{F}$ and its extension field $\mathbb{EF}$, and for all positive integers N, B , there exists a PIOP for the relation $\mathcal{R}_{\text{acc}}$ that achieves perfect completeness and soundness error $O(\ell N)/|\mathbb{EF}|$, with communication cost of $O(\log^2 N + \ell)$. The prover runs in $O(\ell N)$ time, and the verifier in $O(\log^2 N + \ell)$ time.

We prove it with Construction 7 in Appendix C.5.

4.4 Modulus Switching

Modulus switching scales a field element $a \in \mathbb{F}$ with modulus Q to a ring element $b \in \mathbb{Z}_q$ with modulus q . Specifically, the modulus switching function is defined as:

$$b = \left\lfloor \frac{a \cdot q}{Q} \right\rfloor \pmod{q}$$

which rounds the fraction $\frac{a \cdot q}{Q}$ to the nearest integer, followed by a modular reduction modulo q . Notably, $\lfloor \frac{a \cdot q}{Q} \rfloor$ may wrap around modulo q , potentially resulting in $b = 0$.

We consider modulus switching from Q to q , under the assumption that $2q|Q - 1$. This assumption is satisfied under typical FHEW/TFHE parameter settings (see Appendix D.1 for further discussion). Define $k = \frac{Q-1}{2q}$, so that $Q - 1 = 2kq$ holds exactly. A key observation is that the modulus switching under this equality can be interpreted as a structured mapping, as illustrated in Fig. 3. This mapping takes a sequential range in $\mathbb{F}$ and maps it to the same element in $\mathbb{Z}_q$. To handle the special range $[0, k] \cup [Q - k, Q]$, we reformulate it as $\{k\} \cup [(2q - 1) \cdot k + 1, (2q + 1) \cdot k] \pmod{Q}$ using the equality $Q - 1 = 2kq$. By leveraging $b' \in \{1, \dots, q\}$ such that $b \equiv b' \pmod{q}$, we can represent modulus switching as a highly structured, unified, and parameterized mapping, as described in Theorem 4.4.

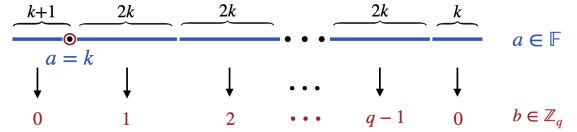


Figure 3: Assuming $2q|Q - 1$ with $k = \frac{Q-1}{2q}$, the modulus switching maps $a \in [0, k] \cup [Q - k, Q]$ to 0, and $a \in [(2b - 1) \cdot k + 1, (2b + 1) \cdot k]$ for each $b \in \{1, \dots, q - 1\}$ to $b \in \mathbb{Z}_q$.

THEOREM 4.4. Let q be a power of two, and let Q be a prime such that $2q|Q - 1$. Define $k = \frac{Q-1}{2q}$ so that $Q = 2kq + 1$. For any $a \in \mathbb{F}_Q$ and $b \in \mathbb{Z}_q$, we have $b = \lfloor \frac{a \cdot q}{Q} \rfloor$ if and only if either $a = k$ and $b = 0$, or there exists $b' \in \{1, \dots, q\}$ such that $b' \equiv b \pmod{q}$ and $a \in [(2b' - 1) \cdot k + 1, (2b' + 1) \cdot k] \subset \mathbb{F}$.

The proof of Theorem 4.4 is provided in Appendix D.2.

Specifically, Theorem 4.4 extracts a dichotomy from the modulus switching relation $\mathcal{R}_{\text{switch}}$ (Definition 11). This dichotomy consists of two cases, each of which can be verified with an equality test. In the first case, where $a = k$ and $b = 0$, we prove correctness by checking that $p = \lambda_1 \cdot (a - k) + \lambda_2 \cdot b = 0$, where $\lambda_1 \xleftarrow{\$} \mathbb{EF}$ and $\lambda_2 \xleftarrow{\$} \mathbb{EF}$. In the second case, we prove that $c = a - (2b' - 1) \cdot k - 1 \in \mathbb{F}$ lies within the range $[0, 2k]$ using the range check protocol based on gadget decomposition, as detailed in Section 3.3. By introducing a binary witness $w \in \{0, 1\}$ to indicate which case applies, we unify both conditions:

$$w \cdot p + (1 - w) \cdot (a - (2b' - 1) \cdot k - 1 - c)$$

where $w = \mathbb{1}(a = k)$ can be computed efficiently. Based on this approach, we propose a PIOP for the modulus switching relation $\mathcal{R}_{\text{switch}}$, involving only Hadamard products and range checks.

DEF 11. The modulus switching relation $\mathcal{R}_{\text{switch}}$ is the set of tuples

$$(i, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, Q, q, N, B, \mathbf{b}), \mathbf{a})$$

such that $\mathbf{a} \in \mathbb{F}_Q^N$ and $\mathbf{b} \in \mathbb{Z}_q^N$, under the assumption that $2q|Q - 1$. Here, $B \in \mathbb{Z}$ is a tunable basis. For each $i \in [N]$, we have $b_i = \lfloor \frac{a_i \cdot q}{Q} \rfloor$.

THEOREM 4.5. For every finite field $\mathbb{F}$, and its extension field $\mathbb{EF}$, and for all positive integers N and B , there is a PIOP for the relation $\mathcal{R}_{\text{switch}}$ that achieves perfect completeness and soundness error $O(\ell N)/|\mathbb{EF}|$, with communication cost of $O(\log N + \ell)$, where $\ell = \lceil \log_B \frac{Q-1}{q} \rceil$. The prover runs in $O(\ell N)$ time, and the verifier in $O(\log N + \ell)$ time.

We prove it with Construction 8 in Appendix C.6.

4.5 PIOP for FHE NAND

We propose a PIOP for the FHE NAND operation with bootstrapping, as described in Algorithm 3 and illustrated in Fig. 1. By invoking the PIOPs for all involved relations—each capturing the correctness of an essential step in the algorithm—we construct a PIOP for the overall NAND relation $\mathcal{R}_{\text{NAND}}$ (Definition 12).

As described in Algorithm 3, it takes as input two LWE ciphertexts ct_0, ct_1 , a constant vector $\mathbf{v}$, and n RGSW ciphertexts $Z_0, \dots, Z_{n-1}$ serving as public bootstrapping keys. It outputs the resulting LWE ciphertext ct along with the computation traces. The traces include n RLWE values and n sub-traces $\{\text{acc}[i], \text{tc}[i]\}_{i=0}^{n-1}$, which record the accumulator states and their corresponding update trace at each update. The detailed components of traces and the sub-traces are defined in Algorithm 2 and Algorithm 3, respectively. The overall computation primarily consists of one LWE addition, $n+1$ NTT operations on monomials, n accumulator updates, and $n+1$ modulus switching operations. The remaining lightweight operations can be captured using Hadamard relations.

Notably, by introducing an appropriate constant auxiliary vector, we can selectively apply different linear operations to individual entries of a vector. For example, using the auxiliary vector $\text{aux}_b = (0, \dots, 0, 1) \in \mathbb{Z}_2^{n+1}$, the Hadamard identity $(q - \mathbf{a}) \cdot (1 - \text{aux}_b) + \mathbf{a} \cdot \text{aux}_b = 0$ holds if and only if $b_i = q - a_i$ for all $i \in [n]$ and $b_n = a_n$. This establishes the relation for verifying the correctness of Step 3 through 6. Similarly, Step 19 applies modulus switching selectively to the single entry $\text{acc}.b_0$ using an auxiliary vector. Further details can be found in Construction 9 of Appendix C.7.

DEF 12. The NAND relation $\mathcal{R}_{\text{NAND}}$ is the set of tuples $(\mathbf{i}, \mathbf{x}, \mathbf{w}) =$

$$((\mathbf{v}, \{Z_i\}_{i=0}^{n-1}), (\mathbb{F}, \mathbb{EF}, N, n, q, B, \ell, \text{ct}_0, \text{ct}_1, \text{ct}), \text{traces})$$

where $\ell = \log_B Q$, $\text{ct}_0, \text{ct}_1, \text{ct} \in \text{LWE}$, $\mathbf{v} \in \mathbb{F}^N$, $Z_0, \dots, Z_{n-1} \in \text{RGSW}$ and $(\text{ct}, \text{traces}) = \text{HomNAND}(\text{ct}_0, \text{ct}_1, \mathbf{v}, Z_0, \dots, Z_{n-1}, B)$. The components of traces are defined in Algorithm 3.

THEOREM 4.6. For every finite field $\mathbb{F}$, and its extension field $\mathbb{EF}$, and for all positive integers N, n, B , there exists a PIOP for the relation $\mathcal{R}_{\text{NAND}}$ that achieves perfect completeness and soundness error $O(\ell N n) / |\mathbb{EF}|$ with communication cost $O(\log^2 N + \ell n)$ where $\ell = \lceil \log_B Q \rceil$. The prover runs in $O(\ell N n)$ time, and the verifier in $O(\log^2 N + \ell n)$.

We prove it with Construction 9 in Appendix C.7.

Complexity and Security Analysis. We focus on the arguments for proving the n accumulator update relations, as they constitute the most computationally intensive part of the algorithm. In total, this part comprises $n \cdot (2\ell + 2)$ NTT/INTT operations, $2n$ gadget decompositions with $2n\ell$ decomposed gadgets, and $n \cdot (4\ell + 2)$ Hadamard products. Using the batching technique described in Section 3.2, the prover for the $n \cdot (2\ell + 2)$ NTT relations runs in $O(\ell N n)$ time, and the verifier in $O(\log^2 N + \ell n)$, where the ℓn term accounts for evaluation queries on the original coefficient vectors. These queries are used to check the random linear combination between the randomized NTT relation and the original $O(\ell n)$ NTT relations. This batched NTT argument contributes a soundness error of $O(\log^2 N) / |\mathbb{EF}|$. The $2n$ gadget decompositions are actually reduced to a single batched lookup argument over $2n\ell$ vectors of length N , using the lookup table $\mathbf{t} = (0, \dots, B-1)$. According to

Algorithm 3 HomNAND($\text{ct}_0, \text{ct}_1, \mathbf{v}, Z_0, \dots, Z_{n-1}$)

Input: $\text{ct}_0, \text{ct}_1 \in \text{LWE}$, and $B \in \mathbb{Z}$

Input: $\mathbf{v} \in \mathbb{F}^N$ and $Z_0, \dots, Z_{n-1} \in \text{RGSW}$

Output: $\text{ct} \in \text{LWE}$ and traces

```

1:  $\mathbf{a} = \text{ct}_0 + \text{ct}_1 \pmod{q}$  // LWE Addition
2: // Bootstrapping
3: for  $i = 0$  to  $n - 1$  do
4:    $b_i = q - a_i$ 
5: end for
6:  $b_n = a_n$ 
7:  $\mathbf{c} = (\mathbf{b} \cdot \frac{2N}{q}) \in \mathbb{Z}_{2N}^{n+1}$ 
8: for  $i = 0$  to  $n$  do // Mono-NTT
9:    $\mathbf{d}_i = \text{NTT}(X^{c_i} \pmod{X^N + 1})$ 
10: end for
11:  $\text{acc}[0] = (0, \mathbf{v} \circ \mathbf{d}_n) \in \text{RLWE}$ 
12: for  $i = 0$  to  $n - 1$  do // Accumulator Update
13:   //  $\text{acc}[i+1] = \text{acc}[i] + (\mathbf{d}_i - 1) \otimes \text{acc}[i] \diamond Z_i$ 
14:    $(\text{acc}[i+1], \text{tc}[i]) \leftarrow \text{AccUpdate}(\text{acc}[i], \mathbf{d}_i - 1, Z_i, B)$ 
15: end for
16: for  $i = 0$  to  $n - 1$  do // Modulus Switching
17:    $\mathbf{a}'_i = \lfloor \frac{(\text{acc}[n].a_i \cdot q)}{Q} \rfloor$ 
18: end for
19:  $\mathbf{b}' = \lfloor \frac{(\text{acc}[n].b_0 + 8/Q) \cdot q}{Q} \rfloor$ 
20: return  $\text{ct} = (\mathbf{a}', \mathbf{b}')$  and  $(\mathbf{a}, \mathbf{b}, \{\mathbf{d}_i\}_{i=0}^n, \{\text{acc}[i], \text{tc}[i]\}_{i=0}^{n-1})$ 

```

Theorem 3.4, the prover runs in $O(\ell N n)$, and both the verifier and the communication cost are $(\log N + n\ell)$. This sub-protocol incurs the dominant soundness error of $O(\ell N n) / |\mathbb{EF}|$. Since the protocol for proving Hadamard products is essentially a sum-check protocol, we can batch these sum-check sub-protocols with a random linear combination. As a result, the overall prover runtime remains $O(\ell N n)$, and both the verifier runtime and the communication cost are $O(\log^2 N + \ell n)$. The total soundness error of $\mathcal{R}_{\text{NAND}}$ argument is dominated by the batched lookup argument for gadget decomposition and equals $O(\ell N n) / |\mathbb{EF}|$. The asymptotic complexity of all PIOPs for the internal FHE operations and the NAND operation is summarized in Table 4 of the Appendix.

5 HASTEBOOTS: SNARGS FOR FHE NAND

By using the cryptographic compiler in [20] to combine the PIOP for FHE NAND in Section 4.5 and the Brakedown PCS[41], we build HasteBoots, a succinct argument system that can efficiently generate proofs of FHE NAND with bootstrappings.

In our setting, given the bootstrapping key $\{Z_i\}_{i=0}^{n-1}$ and the constant vector $\mathbf{v}$, both the prover and the verifier hold two public input LWE ciphertexts, ct_0 and ct_1 along with the output LWE ciphertext ct . The prover additionally holds the computation trace of the FHE NAND as the witness, enabling succinct verification time and communication. However, there is no need to protect the witness data, since the FHE circuit and the input (output) ciphertexts are all public. Under this setup, a SNARG system is sufficient without requiring knowledge extractors (i.e., SNARKs) nor requiring zero-knowledge; thus, there is no need for knowledge assumptions or masking polynomials, which improves the concrete efficiency.

5.1 Definitions

Formally speaking, let the index $\mathbf{i}$ contain bootstrapping keys and the constant vector, and the public input $\mathbf{x} = (\text{ct}_0, \text{ct}_1, \text{ct})$ be the LWE ciphertexts. The witness $\mathbf{w} = \text{traces}$ is computation traces defined in Algorithm 3. HasteBoots is a tuple of algorithms (PPGen, KeyGen, Prove, Verify). We give the formal definitions in Appendix F.

- $\text{pp} \leftarrow \text{PPGen}(1^\lambda)$: Given the security parameter λ , this algorithm generates the public parameters pp .
- $(\text{ipk}, \text{ivk}) \leftarrow \text{KeyGen}(\mathbf{i}, \text{pp})$: Given the index $\mathbf{i}$, this algorithm generates a proving key ipk and a verification key ivk .
- $\pi \leftarrow \text{Prove}(\mathbf{x}, \text{ipk})$: Given the public input $\mathbf{x}$ and the proving key ipk , the algorithm generates the proof π .
- $\{0, 1\} \leftarrow \text{Verify}(\mathbf{x}, \text{ivk}, \pi)$: Given the public input $\mathbf{x}$ and the verifying key ivk , the algorithm verify the proof π .

5.2 Polynomial Commitment Scheme

Polynomial commitment schemes (PCS) enable a prover to commit to a polynomial and later prove its evaluations at chosen points. They are widely used to implement oracles in PIOPs. This paper focuses on multilinear PCS, particularly Brakedown [41], which achieves a theoretically optimal linear-time prover with practical efficiency. For details on definitions and constructions, see [41].

We describe our optimization of packing small polynomials, as a single FHE NAND procedure involves many small polynomials.

Packing Small Polynomials. The standard polynomial commitment schemes provide functionality to commit and query a single polynomial. In HasteBoots, we often need to query multiple polynomials at the same points. While pairing-based polynomial commitment schemes have group elements as their commitments, whose homomorphism improves the efficiency by batching the commitments and the queries, the hash-based polynomial commitment schemes such as Brakedown have hash value as their commitments, do not have significant advantages by batching. However, we observe that Brakedown PCS has poor performance for small polynomials; thus, we consider packing multiple small polynomials into a large one to improve efficiency.

Considering M small multilinear polynomials $c_0(y), \dots, c_{M-1}(y)$, where $y \in \{0, 1\}^{\log N}$, we can pack these polynomials into a large multilinear polynomial $C(x, y)$ such that for each $X \in [M]$, it holds that $C(x, y) = c_X(y)$ for every $y \in \{0, 1\}^{\log N}$. This large polynomial can be viewed as an $N \times M$ matrix, where the $\mathcal{Y}$ -th column corresponds to the small multilinear polynomial $c_{\mathcal{Y}}$. Consequently, the prover can only commit a single $(\log N + \log M)$ -variate multilinear polynomial $C(x, y)$. When the verifier wants to query the values of $\tilde{c}_0(r), \dots, \tilde{c}_{M-1}(r)$ at the same point $r \in \mathbb{F}^{\log N}$, the prover first computes $\tilde{c}_i(r)$ for each $i \in [M]$ and sends these values to the verifier. These subclams can be reduced to the evaluations on the matrix $\tilde{C}(r, r')$ via a random linear combination, where $r' \in \mathbb{F}^{\log M}$ is sampled by the verifier. It is sufficient for the verifier to query the evaluation $\tilde{C}(r, r')$ at the joint point. The correctness follows the same idea in Eq. (6) when proving the reduction in the batched NTT on monomials in Section 4.2. This packing introduces soundness error of $\log M / |\mathbb{F}|$ by the Schwartz-Zippel lemma.

FHE Operation		Proving Time	Verification Time	Proof Size
LWE Addition		17 ms	6 ms	11 MB
Bootstrapping	Mono-NTT	151 ms	36 ms	35 MB
	Acc Update	5.02 s	219 ms	107 MB
	Modulus Switching	22 ms	7 ms	12 MB
NAND		5.13 s	219 ms	107 MB

Table 1: Performance of HasteBoots on Apple M4. Results include both PIOP and Brakedown PCS costs.

6 IMPLEMENTATION AND EVALUATION

We implemented HasteBoots as a library in about 33K lines of Rust code. It includes PIOPs for all atomic and FHE operations, an optimized PCS based on Brakedown [41], and a framework for generating SNARGs tailored to FHEW-like schemes. We plan to open-source our library.

Our goal is to determine whether HasteBoots can prove FHE in practice, and thus answer the questions:

- Can HasteBoots generate proofs in seconds?
- What is the main bottleneck in the proof generation?

Experiment Setup. We select FHE parameters to ensure compatibility with our SNARG construction, setting $n = 1024$, $q = 1024$, $N = 1024$, and $Q = 2^{31} - 2^{27} + 1$. Our protocols operate over a 128-bit extension field of size Q^D with $D = 4$ to ensure the soundness, while the underlying FHE instance is defined over a base field of size Q . For gadget decomposition, we set the gadget size to $B = 2^7$. In our implementation of the lookup protocol LogUp [43], we set the batch block size of 3 to minimize overhead. For Brakedown PCS [41], we set $\alpha = 0.1195$, $\beta = 0.0248$, and $r = 1.9$, where α controls the size of the subcode in each recursive call, β defines the code distance, and r adjusts the code rate. We also set a recursion stopping threshold of 10 to optimize performance.

Performance. We evaluate the performance of HasteBoots on an Apple M4, measuring the proving time, verification time and proof size for each FHE operation listed in Table 1. The asymptotic complexity for PIOPs and PCS are given in Appendix G.1. As shown in the results, the bootstrapping procedure is the most time-consuming part in the NAND circuit. Within the bootstrapping procedure, accumulator updates dominate the cost, accounting for 95% of the total time due to its complex workflow. Overall, HasteBoots achieves efficient proving time around 5.13 seconds, verification time of 219 ms, and a proof size of approximately 107 MB.

Proving Time v.s. Proof Size. To better understand the performance trade-offs in our SNARG construction, we isolate the contributions of the PCS and PIOP components in Table 2. The PCS cost consists of the polynomial commitment time and the time to generate evaluation proofs for large committed polynomials, as described in Section 5.2. With the Brakedown PCS, the prover spends around 25% of the total proving time on PCS, while PCS accounts for 95% of the verifier's total time and proof size.

To explore the trade-off between proof size and proving time, we replace the Brakedown PCS with BaseFold [68], using its implementation in Jolt-b [25]. As shown in Table 2, this substitution significantly reduces both the verification time and proof size, albeit at the cost of increased proving time. Nonetheless, even with this trade-off, our total proving time remains faster than the state-of-the-art result—around half an hour—reported in [61].

		Proving Time	Verification Time	Proof Size
NAND w/ Brakedown	PCS	1.39 s	212.5 ms	104 MB
	PIOP	3.75 s	6.9 ms	3 MB
	SNARG	5.13 s	219.4 ms	107 MB
NAND w/ BaseFold	PCS	63.74 s	4 ms	8 MB
	PIOP	3.75 s	6.9 ms	3 MB
	SNARG	67.49 s	10.9 ms	11 MB

Table 2: Comparison using Brakedown PCS vs. BaseFold PCS.

Proving Time	RISC0 [59]	SP1 [58]	[61]	HasteBoots
M3 Pro (8 cores)	–	–	40 min	7 s
C61.meta (128 cores)	–	–	21 min	5 s
Hpc7a.96xlarge (192 cores)	4600 min	1500 min	18 min	4 s
M4 Pro	–	–	–	5 s

Table 3: Proving time for a single bootstrapping procedure across various devices.

Comparison with Prior Work. We compared the performance of HasteBoots with previous work on proving the FHE bootstrapping, as summarized in Table 3. zkVM-based approaches such as RISC0 [59] and SP1 [58], require roughly 3 days and 1 day of proving time, respectively, primarily due to performance losses when compiling complex FHE programs. Thibault and Walter [61] design a SNARK-friendly arithmetic circuit for TFHE bootstrapping procedure and prove it with Plonk [33], reducing proving time to around half an hour. However, as a general-purpose SNARK, Plonk still suffers from some inefficiency when applied FHE-specific operations. In contrast, avoiding proving the full arithmetic circuit, HasteBoots directly prove the relation between the inputs and the outputs of these FHE operations, enabling sub-10 second proof generation.

Concretely, [61] proves an explicit NTT circuit of size $O(N \log N)$, resulting in at least quasi-linear prover time. Our approach, by contrast, proves the NTT relation directly, avoiding circuit encoding and achieving an optimal linear-time prover. Moreover, [61] employs a decomposition method to compute multiplication by a monomial, leading to another circuit of size $O(N \log N)$, while our protocol achieves a linear-time protocol for batched NTT operation on monomials. Finally, [61] introduces additional noise to FHE ciphertexts in modulus switching, which is avoided in our protocol.

[61] achieves a verification time of approximately 8 ms and a proof size of around 200 KB, whereas HasteBoots requires roughly 219 ms and 107 MB. Notably, both the proof size and verification time of HasteBoots can be further optimized by adopting recursion techniques and replacing Brakedown with a more efficient PCS.

7 RELATED WORKS

Fully Homomorphic Encryption. There are two main categories of fully homomorphic encryption schemes. The first category is CKKS[19], BFV[9, 29], and BGV[10], which support the package of multiple messages into a single ciphertext, thus supporting SIMD-like batching homomorphic operations. However, their bootstrapping operations are very heavy. Although these schemes support leveled FHE without bootstrapping, they only support a small class of circuits with a maximum depth smaller than a certain upper bound and are not universal. The second category is FHEW[28], TFHE[21], and variants[6, 24, 46, 49]. They don’t support batching, but they can perform very fast bootstrapping. We focused on

the FHEW scheme with GINX[34] bootstrapping method, since its bootstrapping is lightweight for SNARGs.

While FHE offers strong privacy, it lacks inherent guarantees for integrity. This has led to a line of research [16, 17, 22, 47, 63, 69] on attack that exploit vulnerabilities in ciphertext manipulation, potentially enabling adversaries to alter encrypted data and even recover the client’s secret key.

Succinct Arguments. It has been extensively studied to provide efficient, verifiable proofs for computational integrity. Existing SNARKs [4, 7, 18, 20, 33, 41, 42, 56, 66] are designed for arbitrary computations. Zero-knowledge Virtual Machine (zkVM) [58, 59] build on general-purpose SNARKs to compile high-level language programs into proofs. However, neither general-purpose SNARKs nor zkVMs account for the specific arithmetic structures in FHE, leading to inefficiencies.

Verifiable FHE. While recent research has advanced theoretical constructions for verifiable FHE, practical and efficient solutions remain elusive. Early work [5, 30, 31, 35, 37, 39, 62] supports only basic FHE operations such as LWE additions and omits bootstrapping, thus falling short of enabling fully homomorphism. [12, 52] show that verifiable FHE is possible with SNARKs. [3] provides a theoretical construction by encoding complete FHE using standard R1CS representations, but this method does not natively support the field arithmetic of FHE. To address this, [45] introduces R1CS defined over rings, which aligns with FHE arithmetic. However, these methods offer only theoretical results without practical constructions. Moreover, they rely on general proof techniques, which are not tailored for FHE. In contrast, our work directly proves FHE’s core operations without converting them into generic circuits.

A recent line of work focuses on constructing concrete SNARKs for verifiable FHE. One approach is to use high-level languages, such as Rust, to implement FHE and then compile it into proofs using zkVM frameworks like RISC0 and SP1 [58, 59]. However, this method suffers from extremely low efficiency, with proof generation taking several days. A more refined approach by Thibault and Walter [61] designs a SNARK-friendly arithmetic circuit for TFHE bootstrapping procedure and proves it using Plonk [33], reducing proving time to around half an hour. Yet, general-purpose SNARKs are not optimized for the specialized structure of FHE operations, limiting their performance in this context. In contrast, our work introduces a concrete succinct argument specifically tailored for FHE, enabling proof generation in seconds and achieving significant improvements in both efficiency and practical usability.

Another line of work [2, 14, 15, 36] performs the integrity check on the plaintext instead of checking the computation result of ciphertexts. First, these approaches are not publicly verifiable so only the users with FHE secret keys can decrypt and verify the result. This is not desirable in decentralized applications [11, 44], where the result on the chain needs to be verified by anyone. Second, these approaches are vulnerable to active attacks [23] on FHE.

A potential alternative approach toward verifiable FHE leverages the interactive proofs, such as Vector Oblivious Linear Evaluations (VOLE) [8, 26, 27, 64, 65, 67]. However, VOLE is mainly used to achieve zero-knowledge rather than succinctness, leading to verifier cost at least linear in the prover. However, the verifiable FHE computations are public, so succinctness is more important.

ACKNOWLEDGMENTS

Yu Yu is supported by the National Natural Science Foundation of China (Grant Nos. 92270201 and 62125204), and by the New Corner-stone Science Foundation through the XPLOER PRIZE. Xiang Xie is supported by the project “Research on Verifiable Fully Homomorphic Encryption Algorithms” (Project No. 23511101300), a key technology research initiative on blockchain under the 2023 Shanghai Science and Technology Innovation Action Plan. Yuncong Hu is supported by the National Natural Science Foundation of China, the Science and Technology Commission of Shanghai Municipality, the Shanghai Science and Technology Program (Grant No. 24BC3200200), the ExploreX project of SJTU, and gifts/awards from BIANJIE.AI, Polyhedra, and Xiaomi.

REFERENCES

- [1] Shweta Agrawal and Dongdai Lin (Eds.). 2022. *ASIACRYPT 2022, Part II*. LNCS, Vol. 13792. Springer, Cham.
- [2] Diego F Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. 2023. Heliopolis: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. *ePrint* (2023).
- [3] Shahla Atapoor, Karim Bagheri, Hilder VL Pereira, and Jannik Spiessens. 2024. Verifiable FHE via Lattice-based SNARKs. *ePrint* (2024).
- [4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *ICALP 2018 (LIPIcs, Vol. 107)*. Ioannis Chatzigiannakis, Christos Kaklamani, Dániel Marx, and Donald Sannella (Eds.). Schloss Dagstuhl, 14:1–14:17. <https://doi.org/10.4230/LIPIcs.ICALP.2018.14>
- [5] Alexandre Bois, Ignacio Cascudo, Dario Fiore, and Dongwoo Kim. 2021. Flexible and efficient verifiable computation on encrypted data. In *PKC*.
- [6] Charlotte Bonte, Ilia Iliashenko, Jeongeun Park, Hilder V. L. Pereira, and Nigel P. Smart. 2022. FINAL: Faster FHE Instantiated with NTRU and LWE, See [1], 188–215. https://doi.org/10.1007/978-3-031-22966-4_7
- [7] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orru. 2022. Gemini: Elastic SNARKs for diverse environments. In *EUROCRYPT*.
- [8] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. 2018. Compressing vector OLE. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 896–912.
- [9] Zvika Brakerski. 2012. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *CRYPTO 2012 (LNCS, Vol. 7417)*, Reihaneh Safavi-Naini and Ran Canetti (Eds.). Springer, Berlin, Heidelberg, 868–886. https://doi.org/10.1007/978-3-642-32009-5_50
- [10] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2012. (Leveled) fully homomorphic encryption without bootstrapping. In *ITCS 2012*, Shafi Goldwasser (Ed.). ACM, 309–325. <https://doi.org/10.1145/2090236.2090262>
- [11] Benedikt Bünz, Shashank Agrawal, Mahdi Zamani, and Dan Boneh. 2020. Zether: Towards privacy in a smart contract world. In *FC*.
- [12] Sébastien Canard, Caroline Fontaine, Duong Hieu Phan, David Pointcheval, Marc Renard, and Renaud Sirdey. 2024. Relations among new CCA security notions for approximate FHE. *ePrint* (2024).
- [13] Anne Canteaut and Yuval Ishai (Eds.). 2020. *EUROCRYPT 2020, Part I*. LNCS, Vol. 12105. Springer, Cham.
- [14] Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. 2022. Verifiable encodings for secure homomorphic analytics. *arXiv:2207.14071* (2022).
- [15] Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. 2023. Poster: Verifiable Encodings for Maliciously-Secure Homomorphic Encryption Evaluation. In *CCS*.
- [16] Bhuvnesh Chaturvedi, Anirban Chakraborty, Ayantika Chatterjee, and Debdeep Mukhopadhyay. 2023. vr²FHE- Securing FHE from Reaction-based Key Recovery Attacks. *Cryptology ePrint Archive*, Report 2023/561. <https://eprint.iacr.org/2023/561>
- [17] Bhuvnesh Chaturvedi, Anirban Chakraborty, Ayantika Chatterjee, and Debdeep Mukhopadhyay. 2024. “Ask and Thou Shall Receive”: Reaction-Based Full Key Recovery Attacks on FHE. In *ESORICS 2024, Part IV (LNCS, Vol. 14985)*, Joaquín García-Alfaro, Rafal Kozik, Michał Choraś, and Sokratis Katsikas (Eds.). Springer, Cham, 457–477. https://doi.org/10.1007/978-3-031-70903-6_23
- [18] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *EUROCRYPT 2023, Part II (LNCS, Vol. 14005)*, Carmit Hazay and Martijn Stam (Eds.). Springer, Cham, 499–530. https://doi.org/10.1007/978-3-031-30617-4_17
- [19] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *ASIACRYPT 2017, Part I (LNCS, Vol. 10624)*, Tsuyoshi Takagi and Thomas Peyrin (Eds.). Springer, Cham, 409–437. https://doi.org/10.1007/978-3-319-70694-8_15
- [20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. 2020. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS, See [13], 738–768. https://doi.org/10.1007/978-3-030-45721-1_26
- [21] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology* 33, 1 (Jan. 2020), 34–91. <https://doi.org/10.1007/s00145-019-09319-x>
- [22] Ilaria Chillotti, Nicolas Gama, and Louis Goubin. 2016. Attacking FHE-based applications by software fault injections. *Cryptology ePrint Archive*, Report 2016/1164. <https://eprint.iacr.org/2016/1164>
- [23] Ilaria Chillotti, Nicolas Gama, and Louis Goubin. 2016. Attacking FHE-based applications by software fault injections. *ePrint* (2016).
- [24] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2021. Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE. In *ASIACRYPT 2021, Part III (LNCS, Vol. 13092)*, Mehdi Tibouchi and Huaxiong Wang (Eds.). Springer, Cham, 670–699. https://doi.org/10.1007/978-3-030-92078-4_23
- [25] Cysic Labs. 2024. jolt-b: A High-Performance Implementation of the Jolt Protocol. <https://github.com/cysic-labs/jolt-b>. Commit abc1234, accessed April 13, 2025.
- [26] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. 2022. Improving line-point zero knowledge: Two multiplications for the price of one. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 829–841.
- [27] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. 2020. Line-point zero knowledge and its applications. *Cryptology ePrint Archive* (2020).
- [28] Léo Ducas and Daniele Micciancio. 2015. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *EUROCRYPT 2015, Part I (LNCS, Vol. 9056)*, Elisabeth Oswald and Marc Fischlin (Eds.). Springer, Berlin, Heidelberg, 617–640. https://doi.org/10.1007/978-3-662-46800-5_24
- [29] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Report 2012/144. <https://eprint.iacr.org/2012/144>
- [30] Dario Fiore, Rosario Gennaro, and Valerio Pastro. 2014. Efficiently verifiable computation on encrypted data. In *CCS*.
- [31] Dario Fiore, Anca Nitulescu, and David Pointcheval. 2020. Boosting verifiable computation on encrypted data. In *PKC*.
- [32] Marc Fischlin and Jean-Sébastien Coron (Eds.). 2016. *EUROCRYPT 2016, Part II*. LNCS, Vol. 9666. Springer, Berlin, Heidelberg.
- [33] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. *Cryptology ePrint Archive*, Report 2019/953. <https://eprint.iacr.org/2019/953>
- [34] Nicolas Gama, Malika Izabachène, Phong Q. Nguyen, and Xiang Xie. 2016. Structural Lattice Reduction: Generalized Worst-Case to Average-Case Reductions and Homomorphic Cryptosystems, See [32], 528–558. https://doi.org/10.1007/978-3-662-49896-5_19
- [35] Chaya Ganes, Anca Nitulescu, and Eduardo Soria-Vazquez. 2023. Rinocchio: SNARKs for ring arithmetic. *JoC* (2023).
- [36] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. 2024. How to prove statements obliviously?. In *CRYPTO*.
- [37] Rosario Gennaro, Craig Gentry, and Bryan Parno. 2010. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *CRYPTO*.
- [38] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *41st ACM STOC*, Michael Mitzenmacher (Ed.). ACM Press, 169–178. <https://doi.org/10.1145/1536414.1536440>
- [39] Shafi Goldwasser, Yael Tauman Kalai, Raluca Ada Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. 2013. How to run Turing machines on encrypted data. In *CRYPTO*.
- [40] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. 2021. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. *Cryptology ePrint Archive*, Paper 2021/1043. <https://eprint.iacr.org/2021/1043>
- [41] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS. In *CRYPTO 2023, Part II (LNCS, Vol. 14082)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, 193–226. https://doi.org/10.1007/978-3-031-38545-2_7
- [42] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments, See [32], 305–326. https://doi.org/10.1007/978-3-662-49896-5_11
- [43] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive* (2022).
- [44] Yuncong Hu, Sam Kumar, and Raluca Ada Popa. 2020. Ghostor: Toward a Secure {Data-Sharing} System from Decentralized Trust. In *NSDI*.
- [45] Mi-Ying Miryam Huang, Baiyu Li, Xinyu Mao, and Jiapeng Zhang. 2024. Fully Homomorphic Encryption with Efficient Public Verification. *ePrint* (2024).
- [46] Yongwoo Lee, Daniele Micciancio, Andrey Kim, Rakyong Choi, Maxim Deryabin, Jieun Eom, and Donghoon Yoo. 2023. Efficient FHEW Bootstrapping with Small Evaluation Keys, and Applications to Threshold Homomorphic Encryption. In *EUROCRYPT 2023, Part III (LNCS, Vol. 14006)*, Carmit Hazay and Martijn Stam

- (Eds.). Springer, Cham, 227–256. https://doi.org/10.1007/978-3-031-30620-4_8
- [47] Zengpeng Li, Steven D. Galbraith, and Chunguang Ma. 2016. Preventing Adaptive Key Recovery Attacks on the GSW Levelled Homomorphic Encryption Scheme. In *ProvSec 2016 (LNCS, Vol. 10005)*, Liqun Chen and Jinguang Han (Eds.). Springer, Cham, 373–383. https://doi.org/10.1007/978-3-319-47422-9_22
- [48] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. Zkcn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2968–2985.
- [49] Zeyu Liu, Daniele Micciancio, and Yuriy Polyakov. 2022. Large-Precision Homomorphic Sign Evaluation Using FHEW/TFHE Bootstrapping. See [1], 130–160. https://doi.org/10.1007/978-3-031-22966-4_5
- [50] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. 1992. Algebraic methods for interactive proof systems. *Journal of the ACM (JACM)* 39, 4 (1992), 859–868.
- [51] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *EUROCRYPT 2010 (LNCS, Vol. 6110)*, Henri Gilbert (Ed.). Springer, Berlin, Heidelberg, 1–23. https://doi.org/10.1007/978-3-642-13190-5_1
- [52] Mark Manulis and Jérôme Nguyen. 2024. Fully homomorphic encryption beyond IND-CCA1 security: Integrity through verifiability. In *EUROCRYPT*.
- [53] Daniele Micciancio and Yuriy Polyakov. 2020. Bootstrapping in FHEW-like Cryptosystems. Cryptology ePrint Archive, Report 2020/086. <https://eprint.iacr.org/2020/086>
- [54] Oded Regev. 2005. On lattices, learning with errors, random linear codes, and cryptography. In *37th ACM STOC*, Harold N. Gabow and Ronald Fagin (Eds.). ACM Press, 84–93. <https://doi.org/10.1145/1060590.1060603>
- [55] Ardianto Satriawan, Rella Mareta, and Hanho Lee. 2024. A Complete Beginner Guide to the Number Theoretic Transform (NTT). *Cryptology ePrint Archive* (2024).
- [56] Srinath Setty. 2020. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Annual International Cryptology Conference*. Springer, 704–737.
- [57] Srinath Setty, Justin Thaler, and Riad Wahby. 2024. Unlocking the lookup singularity with Lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 180–209.
- [58] The SuccinctLabs. [n. d.]. The fastest, most feature-complete zkVM for developers. <https://github.com/succinctlabs/sp1>.
- [59] The RISC Zero Team. [n. d.]. Universal Zero Knowledge. <https://risczero.com/>.
- [60] Justin Thaler. 2013. Time-optimal interactive proofs for circuit evaluation. In *Annual Cryptology Conference*. Springer, 71–89.
- [61] Louis Tremblay Thibault and Michael Walter. 2024. Towards Verifiable FHE in Practice: Proving Correct Execution of TFHE’s Bootstrapping using plonky2. *ePrint* (2024).
- [62] Alexander Viand, Christian Knabenhans, and Anwar Hithnawi. 2023. Verifiable fully homomorphic encryption. *arXiv:2301.07041* (2023).
- [63] Alexander Viand, Christian Knabenhans, and Anwar Hithnawi. 2023. Verifiable Fully Homomorphic Encryption. *arXiv:2301.07041 [cs.CR]* <https://arxiv.org/abs/2301.07041>
- [64] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. 2021. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1074–1091.
- [65] Chenkai Weng, Kang Yang, Zhaomin Yang, Xiang Xie, and Xiao Wang. 2022. AntMan: Interactive zero-knowledge proofs with sublinear communication. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2901–2914.
- [66] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*. Springer, 733–764.
- [67] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. 2021. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2986–3001.
- [68] Hadas Zeilberger, Binyi Chen, and Ben Fisch. 2024. BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes. In *Annual International Cryptology Conference*. Springer, 138–169.
- [69] Zhenfei Zhang, Thomas Plantard, and Willy Susilo. 2012. Reaction Attack on Outsourced Computing with Fully Homomorphic Encryption Schemes. In *ICISC 11 (LNCS, Vol. 7259)*, Howon Kim (Ed.). Springer, Berlin, Heidelberg, 419–436. https://doi.org/10.1007/978-3-642-31912-9_28

A FHE NAND WITH BOOTSTRAPPING

Given two LWE ciphertexts ct_0, ct_1 encrypt two bits with binary secret key $s = (s_0, \dots, s_{n-1})$. We first add these two ciphertexts into

a new ciphertext $ct = (a, b) \in \mathbb{Z}_q^{n+1}$ by adding the components of each LWE ciphertext in $\mathbb{Z}_q$.

In the next step, the components of ct are exponentiated by the polynomial $Y = X^{2N/q}$. Specifically, we compute $Y^{-a_0}, \dots, Y^{-a_{n-1}}$ and Y^b . We assume that $2N$ is divisible by q , which is a standard parameter choice in FHE schemes. This process is referred to as the *lift* step.

Let $\{Z_i\}_{0 \leq i \leq n-1}$ be the bootstrapping key, which are RGSW ciphertexts encrypting s_i . Let v be a constant polynomial used to facilitate the extraction of an LWE ciphertext; its specific form is omitted here as it is not crucial to our design. The accumulator, initialized as $\text{acc}[0] = (0, v \cdot Y^b)$ is iteratively updated as $\text{acc}[i+1] = \text{acc}[i] + Y^{-a_i} \otimes \text{acc}[i] \diamond Z_i$ for $0 \leq i \leq n-1$. At the end of this process, the result is an RLWE ciphertext encrypting $v \cdot Y^{b-\langle a, s \rangle}$.

Let (a', b') be the resulting RLWE ciphertext. The coefficient vector of a' and the constant term of b' are extracted as an LWE ciphertext defined over $\mathbb{Z}_Q$. To convert this into a ciphertext defined over $\mathbb{Z}_q$, a modulus switching procedure is applied to each component. The modulus switching function is defined as: $x \in \mathbb{Z}_Q \rightarrow \lfloor \frac{x \cdot q}{Q} \rfloor \bmod q$.

We note a minor modification from the standard bootstrapping procedure. In our design, we set $n = N$ and define the secret key z in the bootstrapping key as a reordered version of the vector s . This avoids the need for a key switching step, simplifying the overall process. It is straightforward to adapt this design to the standard bootstrapping procedure.

B ADDITIONAL DETAILS ON USEFUL PIOP

B.1 Sum-check Protocol

We describe a seminal interactive proof used in our work, known as the sumcheck protocol, proposed by Lund et al.[50]. Suppose there is an μ -variate low-degree polynomial $\mathcal{G} : \mathbb{F}^\mu \rightarrow \mathbb{F}$ where the degree of each variable in $\mathcal{G}$ is at most ℓ . A verifier $\mathcal{V}$, wishes to verify the following claim by an untrusted prover $\mathcal{P}$:

$$T = \sum_{x \in \{0,1\}^\mu} \mathcal{G}(x_1, x_2, \dots, x_\mu)$$

In the sumcheck protocol, $\mathcal{V}$ interacts with $\mathcal{P}$ over μ rounds. At the end of this interaction, $\mathcal{V}$ outputs $b \in \{0,1\}$. The principal cost to $\mathcal{V}$ is the evaluation of $\mathcal{G}$ at a random point in its domain $r \in \mathbb{F}^\mu$. Throughout this paper, we consider the protocol run over the extension field $\mathbb{EF}$, where verifier $\mathcal{V}$ evaluates $\mathcal{G}$ at a random point $r \in \mathbb{EF}^\mu$. We treat the sumcheck protocol as a mechanism to reduce the claim $\sum_{x \in \{0,1\}^\mu} \mathcal{G} \stackrel{?}{=} T$ to the claim $\mathcal{G}(r) \stackrel{?}{=} e$ and denote this reduction protocol as $e \leftarrow \langle \mathcal{P}(\mathcal{G}), \mathcal{V}(r) \rangle (\mu, \ell, T)$, with the following properties:

- **Completeness.** If $T = \sum_{x \in \{0,1\}^\mu} \mathcal{G}(x)$, then for a correct $\mathcal{P}$ and for all $r \in \{0,1\}^*$, $\Pr[\langle \mathcal{P}(\mathcal{G}), \mathcal{V}(r) \rangle (\mu, \ell, T) = 1] = 1$.
- **Soundness.** If $T \neq \sum_{x \in \{0,1\}^\mu} \mathcal{G}(x)$, then for any $\mathcal{P}^*$ and for all $r \in \{0,1\}^*$, $\Pr[\langle \mathcal{P}(\mathcal{G}), \mathcal{V}(r) \rangle (\mu, \ell, T) = 1] \leq \ell \cdot \mu / |\mathbb{EF}|$.
- **Succinctness.** The proof size is $O(\mu \cdot \ell)$ elements of $\mathbb{EF}$.

B.2 Hadamard Products

We provide a public-coin PIOP in Construction 1 for the Hadamard relation $\mathcal{R}_{\text{sparse}}$, which is basically proven by a sum-check protocol. Specifically, $\sum_{i=0}^{M-1} a_i(x) \circ b_i(x) = c(x)$ for all $x \in \{0,1\}^{\log N}$ holds

Protocols	Proving Time	Verification Time	Soundness Error	Round	Communication
<i>Atomic Operations</i>					
Hadamard Protocol	$O(M_1 N)$	$O(\log N + M_1)$	$O(\log N)$	$O(\log N)$	$O(\log N + M_1)$
Batched Lookups	$O(M_2 N)$	$O(\log N + M_2)$	$O(M_2 N)$	$O(\log N)$	$O(\log N + M_2)$
Sparse Matrix Evaluation	$O(M_3)$	$O(\log M_3)$	$O(M_3)$	$O(\log M_3)$	$O(\log M_3)$
NTT	$O(N)$	$O(\log^2 N)$	$O(\log^2 N)$	$O(\log^2 N)$	$O(\log^2 N)$
Gadget Decomposition	$O(L_1 N)$	$O(\log N + L_1)$	$O(L_1 N)$	$O(\log N)$	$O(\log N + L_1)$
<i>FHE Operations</i>					
LWE Addition	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(\log n)$
Mono-NTT	$O(n + N)$	$O(\log^2 N + \log n)$	$O(\log^2 N + n)$	$O(\log^2 N + \log n)$	$O(\log^2 N + \log n)$
Accumulator Update	$O(L_1 N)$	$O(\log^2 N + L_1)$	$O(L_1 N)$	$O(\log^2 N)$	$O(\log^2 N + L_1)$
Modulus Switching	$O(L_2 n)$	$O(\log n + L_2)$	$O(L_2 n)$	$O(\log n)$	$O(\log n + L_2)$
FHE NAND	$O(L_1 N n)$	$O(\log^2 N + L_1 n)$	$O(L_1 N n)$	$O(\log^2 N + \log n)$	$O(\log^2 N + L_1 n)$

Table 4: Asymptotic complexity of PIOPs used throughout our paper. Proving and verification times, and communication costs are measured in terms of (extension) field operations. Soundness error is given in multiples of $1/|\mathbb{EF}|$. Notation: Let Q be the modulus of $\mathbb{RLWE}$ ciphertexts and q the modulus of $\mathbb{LWE}$ ciphertexts such that $2q \mid (Q - 1)$. Let B be a tunable decomposition base, and define $L_1 = \lceil \log_B Q \rceil$ and $L_2 = \lceil \log_B (\frac{Q-1}{q}) \rceil$. Other parameters include N (vector length), n (dimension of the $\mathbb{LWE}$ vectors), M_1 (number of Hadamard products within the sum), M_2 (number of lookup vectors), M_3 (column count/sparsity of the sparse matrix).

if and only if the multivariate polynomial $\mathcal{F}(t) = \sum_{x \in \{0,1\}^{\log N}} \tilde{e}q(x, t) \cdot (\sum_{i=0}^{M-1} a_i(x) \cdot b_i(x) - c(x))$ is identically zero. Hence, it suffices to check whether $\mathcal{F}(\mu) = 0$ at a randomly sampled $\mu \xleftarrow{\$} \mathbb{EF}^{\log N}$. By the Schwartz-Zippel lemma, this introduces a soundness error of at most $\log N/|\mathbb{EF}|$. Combined with the soundness error of the sum-check protocol, the overall soundness error of this PIOP is $O(\log N/|\mathbb{EF}|)$. Completeness follows from the perfect completeness of the sum-check protocol and the fact that if the Hadamard relation holds, the multivariate polynomial is identically zero, implying $\mathcal{F}(u) = T = \sum_{x \in \{0,1\}^{\log N}} \mathcal{G}(x) = 0$.

CONSTRUCTION 1. We construct a PIOP for the indexed relation $\mathcal{R}_0$. $\mathcal{P}$ takes as input an index $i = \perp$, instance $\mathbf{x} = (\mathbb{F}, \mathbb{EF}, N, M)$, and witness $\mathbf{w} = (\{a_0, \dots, a_{M-1}\}, \{b_0, \dots, b_{M-1}\}, c)$.

1. $\mathcal{V}$ samples a challenge $u \xleftarrow{\$} \mathbb{EF}^{\log N}$ and sends it to $\mathcal{P}$.
2. Let $\mathcal{G}(x) = \tilde{e}q(x, u) \cdot (\sum_{i=0}^{M-1} a_i(x) \cdot b_i(x) - c(x))$, $T = 0$, $\mu = \log N$, $\ell = 3$.
3. $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck protocol $e \leftarrow \langle \mathcal{P}(\mathcal{G}), \mathcal{V}(r) \rangle(\mu, \ell, T)$.
4. $\mathcal{V}$ queries evaluations for $i \in [M]$: $e^{a_i} := a_i(r)$, $e^{b_i} := b_i(r)$ and $e^c := c(r)$, and checks that $\tilde{e}q(x, r_i) \cdot (\sum_{i=0}^{M-1} e^{a_i} \cdot e^{b_i} - e^c) = e$.

The prover runs in linear time with $O(MN)$ computation while the verifier's computation is $O(\log N + M)$ where M comes from query complexity. Since the degree of $\mathcal{G}$ in Step 2 is constant and the round complexity of the sum-check protocol is $O(\log N)$, the total communication cost is $O(\log N + M)$, where M accounts for the number of evaluations the prover sends in Step 4.

B.3 Lookups

Lookup arguments are used to prove that a sequence of values belongs to a table, which is often prescribed. Haböck proposed a lookup argument based on the logarithmic derivative in [43]. Let N be a power-of-two less than the characteristic of field $\mathbb{F}$, i.e. $N < Q$. Given two vectors of field elements $\mathbf{a} \in \mathbb{F}^N$ and $\mathbf{t} \in \mathbb{F}^{N'}$, lookup problem can be written as $\{a_i : i = 1, \dots, N\} \subseteq \{t_j : j = 1, \dots, N'\}$ as sets, with multiples of values removed. For simplicity, we assume $N = N'$. According to the Lemma 5 of [43], we have $\{a_i\} \subseteq \{t_i\}$ as sets if and only if there exists a vector $\mathbf{m} \in \mathbb{F}^N$ such that

$$\sum_{i=1}^N \frac{1}{X + a_i} = \sum_{i=1}^N \frac{m_i}{X + t_i}$$

in the function field $\mathbb{F}[X]$. Let $\text{mp}(\mathbf{a}, z)$ denote the multiplicity of an element z in the vector $\mathbf{a}$, with a similar notation for $\mathbf{t}$. The vector $\mathbf{m}$ can be computed as $m_i = \frac{\text{mp}(\mathbf{a}, t_i)}{\text{mp}(\mathbf{t}, t_i)}$ in probabilistic polynomial time.

We consider the batch lookup arguments proposed in [43]. Let $f_0, \dots, f_{M-1}$ and $\mathbf{t}$ be multivariate functions defined over the Boolean hypercube, i.e., $\{0, 1\}^{\log N} \rightarrow \mathbb{F}$. By Lemma 5 in [43], we have $\bigcup_{i=0}^{M-1} \{f_i(x)\} \subseteq \{\mathbf{t}(x)\}$ as sets if and only if there exists a function $\mathbf{m} : \{0, 1\}^{\log N} \rightarrow \mathbb{F}$ such that

$$\sum_{x \in \{0,1\}^{\log N}} \sum_{i=0}^{M-1} \frac{1}{X + f_i(x)} = \sum_{x \in \{0,1\}^{\log N}} \frac{\mathbf{m}(x)}{X + \mathbf{t}(x)}$$

in the function field $\mathbb{F}[X]$. For each $x \in \{0, 1\}^{\log N}$, the function $\mathbf{m}(x)$ can be computed as $\mathbf{m}(x) = \frac{\sum_{i=0}^{M-1} \text{mp}(f_i, \mathbf{t}(x))}{\text{mp}(\mathbf{t}, \mathbf{t}(x))}$.

Hence, we obtain a PIOP as described in Construction 2 for the relation $\mathcal{R}_{\text{lookup}}$ with perfect completeness and soundness error $O(MN)/|\mathbb{F}|$. The prover runs in $O(MN)$ time, while the verifier's runtime is $O(\log N + M)$. The total communication cost is $O(\log N + M)$. The complexity and security analyses follow from [43].

CONSTRUCTION 2. We construct a PIOP for the indexed relation $\mathcal{R}_{\text{lookup}}$. The prover $\mathcal{P}$ takes as input an index $\mathfrak{i} = \perp$, the public input $\mathfrak{x} = (\mathbb{F}, \mathbb{F}, N, t, M)$, and the witness $\mathfrak{w} = (f_0, \dots, f_{M-1})$.

1. $\mathcal{P}$ computes the multiplicity polynomial $\mathbf{m}(x) = \frac{\sum_{i=0}^{M-1} \text{mp}(f_i, t(x))}{\text{mp}(t, t(x))}$ and sends the oracle message to $\mathcal{V}$.
2. $\mathcal{V}$ samples a random challenge $\rho \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.
3. $\mathcal{P}$ compute $\mathbf{h}_i(x) = \frac{\mathbf{m}_i(x)}{\varphi_i(x)}$ where $\mathbf{m}_i(x) = -1$ and $\varphi_i(x) = \rho + f_i(x)$ for $i \in [M]$ and $\mathbf{m}_M(x) = \mathbf{m}(x)$ and $\varphi_M(x) = \rho + t(x)$. $\mathcal{P}$ sends the oracle messages $\mathbf{h}_0, \dots, \mathbf{h}_M$ to $\mathcal{V}$.
4. Let $\mathcal{G} = \sum_{i=0}^M \mathbf{h}_i(x)$, $T = 0$, $\mu = \log N$, $\ell = 1$.
5. $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck protocol $e \leftarrow \langle \mathcal{P}(\mathcal{G}), \mathcal{V}(r) \rangle (\mu, \ell, T)$ where $r \in \mathbb{F}^\mu$.
6. $\mathcal{V}$ makes the evaluation queries for $i \in [M+1]$: $e^{\mathbf{h}_i} = \mathbf{h}_i(r)$, and check that $\sum_{i=0}^M e^{\mathbf{h}_i} = e$.
7. For $i \in [M+1]$, $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol in parallel with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{F}, N, 1), (\mathbf{h}_i, \varphi_i, \mathbf{m}_i))$

Moreover, an indexed lookup construction can be readily obtained by applying a public-coin hash function to the index and the table value, introducing only a negligible soundness error. The PIOP for the relation $\mathcal{R}_{\text{id-lookup}}$ is omitted here and has perfect completeness and a soundness error $O(N)/|\mathbb{F}|$. The prover runs in $O(N)$ time, while the verifier's runtime is $O(\log N)$.

C SECURITY PROOF

C.1 Proof of Theorem 3.2

To prove Theorem 3.2, we first provide Construction 3, which defines a public-coin PIOP for sparse matrix evaluation relation $\mathcal{R}_{\text{sparse}}$. This construction is a PIOP derived from [57] with perfect completeness and soundness error of $O(M)/|\mathbb{F}|$, where the memory-checking is simply replaced with an indexed lookup PIOP.

CONSTRUCTION 3. We construct a PIOP for the relation $\mathcal{R}_{\text{sparse}}$. $\mathcal{P}$ takes as input an index $\mathfrak{i} = \perp$, the public input $\mathfrak{x} = (\mathbb{F}, \mathbb{F}, N, M, e, r_x, r_y)$, and the witness $\mathfrak{w} = (\text{val}, \text{row})$; the verifier $\mathcal{V}$ takes as input the index $\mathfrak{i}$ and the instance $\mathfrak{x}$.

1. $\mathcal{P}$ computes $E_{r_x}(k) = \tilde{e}q(\text{to-bits}(\text{row}(k)), r_x)$ for $k \in \{0, 1\}^{\log M}$.
2. Let $\mathcal{G}(k) = \text{val}(k) \cdot E_{r_x}(k) \cdot \tilde{e}q(k, r_y)$, $T = e$, $\mu = \log M$, $\ell = 3$.
3. $\mathcal{P}$ and $\mathcal{V}$ run the sumcheck protocol $e \leftarrow \langle \mathcal{P}(\mathcal{G}), \mathcal{V}(r) \rangle (\mu, \ell, T)$ where $r \in \mathbb{F}^{\log M}$.
4. $\mathcal{V}$ makes the evaluation queries: $e^{\text{val}} := \text{val}(r)$, $e^{E_{r_x}} := E_{r_x}(r)$, and checks that $e^{\text{val}} \cdot e^{E_{r_x}} \cdot \tilde{e}q(r, r_y) = e$.
5. Let $\mathbf{t}(x) = \tilde{e}q(x, r_x)$ for $x \in \{0, 1\}^{\log N}$ and $\text{id}(k) = \text{row}(k)$ for $k \in \{0, 1\}^{\log M}$. For simplicity, assuming $N \leq M$, let $\mathbf{t}'(k)$ be the padded table of $\mathbf{t}(x)$ where $k \in \{0, 1\}^{\log M}$.
6. $\mathcal{P}$ and $\mathcal{V}$ check that E_{r_x} is well-formed using indexed lookup with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{F}, M, \mathbf{t}'), (\text{id}, E_{r_x}))$

Step 1 and 5 can be computed in linear time using a dynamic programming algorithm to compute $\tilde{e}q(\cdot, r_x)$ over the hypercube.

With a linear-time prover for the sum-check protocol and a linear-time prover for the indexed lookup protocol, the prover runs in $O(M)$ time, and the verifier's runtime is $O(\log M)$. The total communication cost is $O(\log M)$.

C.2 Proof of Theorem 3.4

To prove Theorem 3.4, we first provide Construction 4, which defines a public-coin PIOP for the gadget decomposition relation $\mathcal{R}_{\text{dec}}$ with perfect completeness and a soundness error of $O(\ell N)/|\mathbb{F}|$. The core of this construction reduces to batched lookup arguments for proving the range of ℓ gadgets with a soundness error $O(\ell N)/|\mathbb{F}|$. To verify equality of $\sum_{i=0}^{\ell-1} B^i \cdot \tilde{a}_i(x) = \tilde{a}(x)$, it suffices to evaluate at a random point since both sides are multilinear in x , introducing a soundness error of at most $\log N/|\mathbb{F}|$.

CONSTRUCTION 4. We construct a PIOP for the indexed relation $\mathcal{R}_{\text{dec}}$. The prover $\mathcal{P}$ takes as input an index $\mathfrak{i} = \perp$, the public input $\mathfrak{x} = (\mathbb{F}, \mathbb{F}, N, B, \ell)$, and the witness $\mathfrak{w} = (\mathbf{a}, (\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1}))$.

1. $\mathcal{P}$ and $\mathcal{V}$ sets $\mathbf{t} = (0, \dots, B-1, 0, \dots, 0) \in \mathbb{F}^N$ and run the batched lookup protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{F}, N, N, \mathbf{t}), (\mathbf{a}_0, \dots, \mathbf{a}_{\ell-1}))$.
2. $\mathcal{V}$ samples $r \xleftarrow{\$} \mathbb{F}^{\log N}$ and makes the queries: $e^{\mathbf{a}} := \mathbf{a}(r)$, $e^{\mathbf{a}_0} := \mathbf{a}_0(r), \dots, e^{\mathbf{a}_{\ell-1}} := \mathbf{a}_{\ell-1}(r)$, and checks that $\sum_{i=0}^{\ell-1} B^i \cdot e^{\mathbf{a}_i} = e^{\mathbf{a}}$.

The prover's runtime is dominated by the batched lookup reductions, resulting in $O(\ell N)$ extension field operations. The verifier's runtime is $O(\log N + \ell)$.

C.3 Proof of Theorem 4.1

To prove Theorem 4.1, we first introduce Construction 5, which defines a public-coin PIOP for LWE addition relation $\mathcal{R}_{\text{add}}$ with perfect completeness and a soundness error of $O(N)/|\mathbb{F}|$. The soundness error can be easily analyzed through the following construction.

CONSTRUCTION 5. We construct a PIOP for the relation $\mathcal{R}_{\text{add}}$. The prover $\mathcal{P}$ takes as input an index $\mathfrak{i} = \perp$, the public input $\mathfrak{x} = (q, N, \mathbf{a}, \mathbf{b})$, and the witness $\mathfrak{w} = \mathbf{c}$.

1. Define a table $\mathbf{t} = (0, \dots, q-1, 0, \dots, 0)$, and $\mathcal{P}$ and $\mathcal{V}$ run the lookup protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{F}, N, \mathbf{t}, 1), \mathbf{c})$
2. $\mathcal{P}$ computes for all $x \in \{0, 1\}^{\log N}$: $\mathbf{k}(x) = \mathbb{1}(\mathbf{a}(x) + \mathbf{b}(x) \geq q)$ and sends it to $\mathcal{V}$ as an oracle.
3. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{F}, N, 1), (\{\mathbf{k}\}, \{1 - \mathbf{k}\}, 0))$.
4. $\mathcal{V}$ samples $r \xleftarrow{\$} \mathbb{F}^{\log N}$ and makes the queries: $e^{\mathbf{a}} := \mathbf{a}(r)$, $e^{\mathbf{b}} := \mathbf{b}(r)$, $e^{\mathbf{c}} := \mathbf{c}(r)$, $e^{\mathbf{k}} := \mathbf{k}(r)$ and checks that $e^{\mathbf{a}} + e^{\mathbf{b}} = e^{\mathbf{c}} + q \cdot e^{\mathbf{k}}$.

Since Step 2 requires only linear time, and both the lookup and Hadamard protocols have linear-time provers, the prover's overall runtime is $O(N)$. The verifier's runtime is $O(\log N)$.

C.4 Proof of Theorem 4.2

To prove Theorem 4.2, we first provide Construction 6, which defines a public-coin PIOP for the Mono-NTT relation $\mathcal{R}_{\text{Mono-NTT}}$ with perfect completeness and soundness error of $O(\log^2 N + M)/|\mathbb{F}|$. Correctness follows from the linearity of NTT operations and the fact of the query on $\mathbf{c}'$ corresponds exactly to an evaluation of the sparse coefficient matrix at the joint point $\tilde{\mathbf{c}}'(r)$, as shown in Eq. (6).

As for the soundness, if any of the M NTT instances is incorrect, the probability that the corresponding randomized NTT instance appears correct is at most $\log M/|\mathbb{EF}|$, by the Schwartz-Zippel lemma. Applying a union bound to account for this error and the soundness error of NTT protocol to check the single randomized NTT instance, as well as the sparse matrix evaluation protocol, we conclude the overall soundness error is bounded by $O(M + \log^2 N)/|\mathbb{EF}|$.

CONSTRUCTION 6. We construct a PIOP for the relation $\mathcal{R}_{\text{Mono-NTT}}$. $\mathcal{P}$ takes as input an index $\mathfrak{i} = \perp$, the public input $\mathfrak{x} = (\mathbb{F}, \mathbb{EF}, N, M, u)$, and the witness $\mathfrak{w} = (\mathbf{k}, \mathbf{r}, \mathbf{e})$.

1. $\mathcal{V}$ samples $\mathbf{r}' \xleftarrow{\$} \mathbb{EF}^{\log M}$ and queries: $\mathbf{e}' := \mathbf{e}(\mathbf{r}')$
2. $\mathcal{P}$ computes the coefficient vector $\mathbf{c}' \in \mathbb{EF}^N$ of the randomized polynomial $\sum_{y \in \{0,1\}^{\log M}} \tilde{e}q(y, \mathbf{r}') \cdot (\mathbf{k}_y \cdot X^{\mathbf{y}})$.
3. $\mathcal{P}$ and $\mathcal{V}$ run the NTT protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, F^R, N, u, \mathbf{e}'), (\mathbf{c}'))$ and return the subclaim $d \stackrel{?}{=} \tilde{c}'(r)$ for a random point $r \in \mathbb{EF}^{\log N}$.
4. $\mathcal{P}$ and $\mathcal{V}$ run the sparse matrix evaluation protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, M, d, \mathbf{r}, \mathbf{r}'), (\mathbf{k}, \mathbf{r}))$.

In Step 2, all values $\tilde{e}q(y, \mathbf{r}')$ for $y \in \{0,1\}^{\log M}$ can be jointly precomputed in $O(M)$ time using a dynamic programming algorithm. Since each term in the sum is a monomial with only one non-zero coefficient, the coefficient vector $\mathbf{c}'$ can be computed in $O(M)$ time. Using the linear-time prover for both the NTT and the sparse matrix evaluation protocol, the total prover runtime is $O(N + M)$. The verifier runs in $O(\log^2 N + \log M)$ time.

C.5 Proof of Theorem 4.3

The accumulator update involves only Hadamard products, NTT, and gadget decomposition, along with simple linear operations. Hence, Construction 7 defines a public-coin PIOP for the accumulator update relation $\mathcal{R}_{\text{acc}}$, reducing to arguments for the relation corresponding to these operations. Perfect completeness follows from the correctness of the computation. By applying a union bound to the soundness error introduced by each argument, we conclude that the overall soundness error is $O(\ell N)/|\mathbb{EF}|$.

CONSTRUCTION 7. We construct a PIOP for the relation $\mathcal{R}_{\text{acc}}$. The prover $\mathcal{P}$ takes as input an index $\mathfrak{i} = Z$, the public input $\mathfrak{x} = (\mathbb{F}, \mathbb{EF}, F^R, N, B, \ell)$, and the witness $\mathfrak{w} = (\text{acc}, \mathbf{d}, \text{acc}', \text{trace})$.

1. $\mathcal{P}$ unties $(\text{mult}, \text{Imult}, \text{bits}, \text{Nbits}, \text{prod}) := \text{trace}$.
2. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) =$
 - $(\perp, (\mathbb{F}, \mathbb{EF}, N, 1), (\mathbf{d}, \text{acc}.\mathbf{a}, \text{mult}.\mathbf{a}))$
 - $(\perp, (\mathbb{F}, \mathbb{EF}, N, 1), (\mathbf{d}, \text{acc}.\mathbf{b}, \text{mult}.\mathbf{b}))$
 - $(\perp, (\mathbb{F}, \mathbb{EF}, N, 2\ell), (\text{Nbits}[0..2\ell], Z.\tilde{\mathbf{a}}, \text{prod}.\mathbf{a}))$
 - $(\perp, (\mathbb{F}, \mathbb{EF}, N, 2\ell), (\text{Nbits}[0..2\ell], Z.\tilde{\mathbf{b}}, \text{prod}.\mathbf{b}))$

In the subclaim, $\mathcal{V}$ queries at a random point, obtaining the evaluations at point $u \xleftarrow{\$} \mathbb{EF}^{\log N}$: $e^{\text{mult}.\mathbf{a}} = \text{mult}.\mathbf{a}(u)$, $e^{\text{mult}.\mathbf{b}} = \text{mult}.\mathbf{b}(u)$, $e_i = \text{Nbits}[i](u)$ for each $i \in [2\ell]$. These queried values, along with the random point, are then used to instantiate the subsequent NTT protocol.

// Note that the query $e^{\mathbf{d}} = \mathbf{d}(u)$ can also be used to instantiate the batched NTT for monomials (Mono-NTT protocol) in Construction 9 for $\mathcal{R}_{\text{NAND}}$.
3. $\mathcal{P}$ and $\mathcal{V}$ run the NTT protocol using the batching technique with
 - $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, F^R, u, e^{\text{mult}.\mathbf{a}}), (\text{Imult}.\mathbf{a}))$

- $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, F^R, u, e^{\text{mult}.\mathbf{b}}), (\text{Imult}.\mathbf{b}))$
 - $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, F^R, u, e_i), (\text{bits}[i]))$ for $i \in [2\ell]$
4. $\mathcal{P}$ and $\mathcal{V}$ run the gadget decomposition protocol with
 - $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, B, \ell), (\text{Imult}.\mathbf{a}, \text{bits}[0..\ell]))$
 - $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, B, \ell), (\text{Imult}.\mathbf{b}, \text{bits}[\ell..2\ell]))$
 5. $\mathcal{V}$ samples a random point and check $e^{\text{acc}'.\mathbf{a}} = e^{\text{acc}.\mathbf{a}} + e^{\text{prod}.\mathbf{a}}$, $e^{\text{acc}'.\mathbf{b}} = e^{\text{acc}.\mathbf{b}} + e^{\text{prod}.\mathbf{b}}$ with corresponding queries.

Since each argument has a linear-time prover, the overall prover runtime is $O(\ell N)$. Notably, the $2\ell + 2$ NTT instances are verified using the batching technique described in Section 3.2, allowing the verifier to run in $O(\log^2 N + \ell)$ time, where ℓ comes from the query complexity. The total communication cost is also $O(\log^2 N + \ell)$.

C.6 Proof of Theorem 4.5

To prove Theorem 4.5, we first provide Construction 8, which defines a public-coin PIOP for the modulus switching relation $\mathcal{R}_{\text{switch}}$ with perfect completeness and a soundness error of $O(\ell N)/|\mathbb{EF}|$. Perfect completeness follows from Theorem 4.4, which establishes a dichotomy where each case reduces to an equality test. These two equality tests are combined in Step 6 via a sum of two Hadamard products with the indicator id_a . This indicator, defined in Step 1, ensuring that $\text{id}_a(x) = 1$ if and only if $\mathbf{a}(x) = k$. For any x where $\text{id}_a(x) = 1$, we have $\mathbf{a}(x) = k$ and $\mathbf{b}(x) = 0$, which be checked using an equality test with a random linear combination in Step 5, introducing only a negligible soundness error. Otherwise, it reduces to a range check in Step 3, ensuring that $\mathbf{c}(x) \in [0, 2k)$, where $\mathbf{c}(x) = \mathbf{a}(x) - (2\mathbf{b}'(x) - 1) \cdot k - 1$ for any x where $\text{id}_a(x) = 0$. Since $2k = \frac{Q-1}{q}$ is as large as the field when q is relatively small, the range check argument basically reduces to a gadget decomposition argument as described in Section 3.3. This introduces the dominate soundness error of $O(\ell N)$ in the this PIOP. The remaining steps check the relation of $\mathbf{b}' \equiv \mathbf{b} \pmod q$ with a soundness error at most $O(\log N)/|\mathbb{EF}|$. Hence, we conclude that the overall soundness error is $O(\ell N)/|\mathbb{EF}|$.

CONSTRUCTION 8. We construct a PIOP for the relation $\mathcal{R}_{\text{switch}}$. The prover $\mathcal{P}$ takes as input an index $\mathfrak{i} = \perp$, the public input $\mathfrak{x} = (\mathbb{F}, \mathbb{EF}, Q, q, N, B, \mathbf{b})$, and the witness $\mathfrak{w} = \mathbf{a}$.

1. Let $k = \frac{Q-1}{2q}$ and $\ell = \lceil \log_B(2k) \rceil$.
2. $\mathcal{P}$ computes $\text{id}_a, \text{id}_b, \mathbf{b}'$ and $\mathbf{c}$ for each $x \in \{0,1\}^{\log N}$:
 - $\text{id}_a(x) = \mathbb{1}(\mathbf{a}(x) = k)$
 - $\text{id}_b(x) = \mathbb{1}(\mathbf{b}(x) = 0)$
 - $\mathbf{b}'(x) = \text{id}_b(x) \cdot q + \mathbf{b}(x)$
 - $\mathbf{c}(x) = (1 - \text{id}_a(x)) \cdot (\mathbf{a}(x) - (2\mathbf{b}'(x) - 1) \cdot k - 1)$

and sends id_a, id_b and $\mathbf{c}$ to the verifier $\mathcal{V}$, and $\mathbf{b}'$ can be queried via $\mathbf{b}$ and id_b .
3. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with
 - $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, 1)(\{\text{id}_a\}, \{1 - \text{id}_a\}, \mathbf{0}))$
 - $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, 1)(\{\text{id}_b\}, \{1 - \text{id}_b\}, \mathbf{0}))$
4. $\mathcal{P}$ and $\mathcal{V}$ run the range check protocol on $[0, 2k)$ with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, B, \ell, 2k), \mathbf{c})$
5. $\mathcal{V}$ samples $\lambda_1 \xleftarrow{\$} \mathbb{EF}$ and $\lambda_2 \xleftarrow{\$} \mathbb{EF}$ and sends them to $\mathcal{P}$.
6. Let $\text{id}_{\text{point}} = \lambda_1 \cdot (\mathbf{a} - k) + \lambda_2 \cdot \mathbf{b}$ be an oracle that can be queried via $\mathbf{a}$ and $\mathbf{b}$.
7. $\mathcal{P}$ and $\mathcal{V}$ run Hadamard protocol with $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) = (\perp, (\mathbb{F}, \mathbb{EF}, N, 2), (\{\text{id}_a, 1 - \text{id}_a\}, \{\text{id}_{\text{point}}, (\mathbf{a} - (2\mathbf{b}' - 1) \cdot k - 1 - \mathbf{c})\}, \mathbf{0}))$.

Step 1 and 5 can be computed in linear time. The dominant computational cost arises in Step 3, where the range check argument reduces to a gadget decomposition. Since ℓ denotes the length of the decomposed gadgets, the prover's runtime is $O(\ell N)$. The verifier's runtime is $O(\log N + \ell)$ where ℓ comes from the query complexity.

C.7 Proof of Theorem 4.6

As illustrated in Fig. 1, the FHE NAND mainly consists of four steps: LWE addition, NTT operations on monomials, accumulator updates and modulus switching. To ensure the correctness of each essential step in Algorithm 3, the PIOP for the FHE NAND relation $\mathcal{R}_{\text{NAND}}$, defined in Construction 9, also incorporates several Hadamard sub-protocols to verify the remaining lightweight operations.

Notably, the n accumulator updates are both the most computationally intensive and the dominant contributor to the soundness error. By applying a union bound over the soundness error introduced by each individual argument, we conclude that the overall soundness error is $O(\ell N n) / |\mathbb{F}|$. The detailed complexity and soundness analysis is given in Section 4.5.

CONSTRUCTION 9. We construct a PIOP for the $\mathcal{R}_{\text{NAND}}$. The prover $\mathcal{P}$ takes as input an index $\mathbf{i} = (v, \{Z_i\}_{i=0}^{n-1})$, the public input $\mathbf{x} = (\mathbb{F}, \mathbb{E}\mathbb{F}, N, n, q, B, \text{ct}_0, \text{ct}_1, \text{ct})$, and the witness $\mathbf{w} = \text{traces}$.

1. $\mathcal{P}$ unties $(\mathbf{a}, \mathbf{b}, \{\mathbf{d}_i\}_{i=0}^n, \{\text{acc}[i], \text{tc}[i]\}_{i=0}^{n-1}) := \text{traces}$
2. $\mathcal{P}$ and $\mathcal{V}$ run the LWE addition protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (q, n+1, \text{ct}_0, \text{ct}_1), \mathbf{a})$.
3. Let $\text{aux}_b = (0, \dots, 0, 1) \in \mathbb{Z}_2^{n+1}$.
4. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, 2), (\{q - \mathbf{a}, \mathbf{a}\}, \{1 - \text{aux}_b, \text{aux}_b\}, \mathbf{b}))$.
5. $\mathcal{P}$ computes $\mathbf{k} \in \{0, 1\}^{n+1}$ and $\mathbf{r} \in \mathbb{Z}_N^{n+1}$ such that $\mathbf{b} \cdot \frac{2N}{q} = \mathbf{k} \cdot \mathbf{q} + \mathbf{r}$ and sends $\mathbf{k}, \mathbf{r}$ to $\mathcal{V}$ as oracles.
6. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, 1), (\{\mathbf{k}\}, \{1 - \mathbf{k}\}, 0))$.
7. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, 1), (v, \mathbf{d}_n, \text{acc}[0], \mathbf{b}))$
8. For $i \in [n]$: $\mathcal{P}$ and $\mathcal{V}$ run the accumulate update protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (Z, (\mathbb{F}, \mathbb{E}\mathbb{F}, F^R, N, B, \ell), (\text{acc}[i], \mathbf{d}_i - 1, \text{acc}[i+1], \text{tc}[i]))$
// Assuming the underlying sum-check protocols in Step 7 and 8 are executed in parallel, the returned subclaims consists of evaluation queries of the form $\tilde{\mathbf{d}}_i(u) \stackrel{?}{=} e_i$ for each $i \in [n+1]$, where $u \in \mathbb{E}\mathbb{F}^{\log N}$ is a shared random point.
9. Let $\mathbf{e}(x) = \sum_{y \in \{0,1\}^{\log(n+1)}} \tilde{e}_q(x, y) \cdot e_i$ and sends it to $\mathcal{V}$.
// The simple equality test between $\mathbf{e}$ and $\mathbf{d}_0, \dots, \mathbf{d}_n$ is omitted.
10. $\mathcal{P}$ and $\mathcal{V}$ run the Mono-NTT protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, n+1, u), (1 - 2\mathbf{k}, \mathbf{r}, \mathbf{e}))$
11. $\mathcal{P}$ and $\mathcal{V}$ run the modulus switching protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, Q, q, N, B, \text{ct}, \mathbf{a}), \text{acc}[n], \mathbf{a})$
12. Let $\text{aux}_{b'} = (1, 0, \dots, 0)$ and $\mathbf{b}' = (\text{ct}, b, 0, \dots, 0)$.
13. $\mathcal{P}$ computes $\mathbf{b}_{\text{zero}} = (\text{acc}[n], b_0 + Q/8, 0, \dots, 0)$ and sends it to $\mathcal{V}$.
14. $\mathcal{P}$ and $\mathcal{V}$ run the Hadamard protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, N, 1), (\{\text{acc}[n], \mathbf{b} + Q/8\}, \{\text{aux}_{b'}, \mathbf{b}_{\text{zero}}\}))$
15. $\mathcal{P}$ and $\mathcal{V}$ run the modulus switching protocol with $(\mathbf{i}, \mathbf{x}, \mathbf{w}) = (\perp, (\mathbb{F}, \mathbb{E}\mathbb{F}, Q, q, N, B, \mathbf{b}'), \mathbf{b}_{\text{zero}})$

D MODULUS SWITCHING

D.1 Assumption of $2q \mid (Q - 1)$

This assumption is easily satisfied under standard FHEW/TFHE parameters. Specifically, $2N \mid (Q - 1)$ due to requirements from the NTT operation, and $q \mid 2N$ as dictated by the lift operation that maps each ring element in the LWE ciphertext to a monomial in the RLWE domain (see Section 4.2 for details). From a practical standpoint, q is typically chosen as a power of two, and Q is selected in the form $Q = 2^\ell - i \cdot (2N) + 1$ for some integer i , where ℓ denotes the bit-length of the prime Q . This ensures that $2q \mid (Q - 1)$, enabling an efficient PIOP for modulus switching in our work.

D.2 Proof of Theorem 4.5

PROOF. First, it is trivial to observe that $a = k \notin [(2b' - 1) \cdot k + 1, (2b' + 1) \cdot k]$ for every $b' \in \{1, \dots, q\}$. In this case, if $a = k$, we have $\lfloor \frac{k \cdot q}{Q} \rfloor = 0$ since $Q - 1 = 2kq$.

Now, consider $a \in \mathbb{F} \setminus \{k\}$, we can rewrite the equality $kq = \frac{1}{2}(Q - 1)$. For any $a \in [(2b' - 1) \cdot k + 1, (2b' + 1) \cdot k] \subseteq \mathbb{F}$, where $b' \in \{1, \dots, q\}$, the lower bound is given by:

$$a \cdot q \geq ((2b' - 1) \cdot k + 1) \cdot q = (2b' - 1) \cdot \frac{Q - 1}{2} + q > b'Q - \frac{1}{2}Q,$$

where the last inequality holds because $2q + 1 - 2b' > 0$ for any $b' \in \{1, \dots, q\}$.

Next, we compute the upper bound, which we handle in two cases:

Case 1: For any $a \in [(2b' - 1) \cdot k + 1, (2b' + 1) \cdot k] \subseteq \mathbb{F}$ where $b' = b \in \{1, \dots, q - 1\}$, we have

$$a \cdot q \leq (2b' + 1) \cdot kq = (b' + \frac{1}{2}) \cdot (Q - 1) < b'Q + \frac{1}{2}Q$$

Thus, we conclude that $\lfloor \frac{a \cdot q}{Q} \rfloor = b' = b$.

Case 2: For any $a \in [0, k) \cup [Q - k, Q]$, which corresponds to $a \in [(2b' - 1) \cdot k + 1, (2b' + 1) \cdot k] \pmod{Q}$ with $b' = q$ and $b = 0$, we consider the following two subcases:

1. If $a \in [0, k)$, we have $a \cdot q < kq = \frac{1}{2}(Q - 1) < \frac{1}{2}Q$.
2. If $a \in [Q - k, Q]$, we have $a \cdot q < q \cdot Q$.

Combining these two subcases, we conclude that $\lfloor \frac{a \cdot q}{Q} \rfloor = 0 = b \pmod{q}$.

Finally, we conclude that $\lfloor \frac{a \cdot q}{Q} \rfloor = b$. The converse direction can be proven similarly, and we omit the details here. $\square$

E NTT/INTT

E.1 Idea of Delegation Protocol

The evaluation for $\tilde{F}^R(u, v)$ can be delegated to the prover through a sequence of sumcheck protocol, following the same Algorithm 1 to compute A_{FR} . We abuse the notation and use $A_{FR}^{(i)} : \{0, 1\}^{i+1} \rightarrow \mathbb{E}\mathbb{F}$ as in [48] to denote the array A_{FR} in the i -th round for $i = 0, \dots, \log N - 1$. Then $\tilde{F}^R(u, v) = \tilde{A}_{FR}^{(\log N - 1)}(v)$, and we can write $A_{FR}^{(i)}(\cdot)$ as an equation of $A_{FR}^{(i-1)}(\cdot)$:

$$A_{FR}^{(i)}(x, b) = A_{FR}^{(i-1)}(x) ((1 - u_i) + u_i \cdot \omega_{i+1}(x, b)) \cdot \omega^{2^i \cdot b}$$

for all $x \in \{0, 1\}^i, b \in \{0, 1\}$, where $\omega_{i+1}(x, b) = \omega_{2^{i+1}}^j$ with $j = X + 2^i \cdot b$. Note that the right-hand side of the above equation is

not multilinear in either x or b , so we introduce a $(i+1)$ -variate identity function to obtain the following multilinear function

$$\tilde{A}_{FR}^{(i)}(x, b) = \sum_{z \in \{0,1\}^i} \sum_{s \in \{0,1\}} \tilde{e}q((x, b), (z, s)) \cdot \tilde{A}_{FR}^{(i-1)}(z) \cdot ((1 - u_i) + u_i \cdot \tilde{\omega}_{i+1}(z, s)) \cdot \omega^{2^i \cdot s}$$

that holds for all $x \in \mathbb{FF}^i, b \in \mathbb{FF}$.

Starting from $\tilde{F}^R(u, v) = \tilde{A}_{FR}^{(\log N-1)}(v)$, the verifier and the prover can reduce its correctness to the evaluation of $\tilde{A}_{FR}^{(i)}(\cdot)$ at a random point through a sumcheck protocol for $i = \log N - 1, \dots, 0$. In the last round as defined in the Step 1 of Algorithm 1, $\tilde{A}_{FR}^{(0)}(\cdot)$ is simply the constant 1. At the end of each sumcheck protocol, the verifier has to evaluate $\tilde{e}q(\cdot)$ and $\tilde{\omega}_{i+1}(\cdot)$ at a random point to obtain $\tilde{A}_{FR}^{(i-1)}$ at the random point for the next sumcheck protocol. By the closed-form definition of multilinear extension, we have $\tilde{\omega}_{i+1}(r) = \sum_{x \in \{0,1\}^{i+1}} \tilde{e}q(r, x) \omega_{2^{i+1}}^j$ for $j = \sum_{k=0}^{i+1} 2^k \cdot x_k$, which equals to $\prod_{k=0}^{i+1} (1 - r_k + r_k \cdot \omega_{2^{i+1}}^{2^k})$.

E.2 Full Derivation of Eq. (5)

$$\tilde{F}^R(u, x) = \sum_{y \in \{0,1\}^{\log N}} \tilde{e}q(u, y) \tilde{F}^R(y, x) \quad (10)$$

$$= \sum_{y \in \{0,1\}^{\log N}} \tilde{e}q(u, y) \omega^{(2^{\log N+1}) \cdot X} \quad (11)$$

$$= \omega^X \cdot \sum_{y \in \{0,1\}^{\log N}} \tilde{e}q(u, y) \omega^{X \cdot 2^{\log N}} \quad (12)$$

$$= \omega^X \cdot \sum_{y \in \{0,1\}^{\log N}} \tilde{e}q(u, y) \omega^{X \cdot (\sum_{i=0}^{\log N-1} 2^{\log N-i} \cdot y_i)} \quad (13)$$

$$= \omega^X \cdot \sum_{y \in \{0,1\}^{\log N}} \prod_{j=0}^{\log N-1} ((1 - u_j)(1 - y_j) + u_j y_j) \quad (14)$$

$$\cdot \prod_{i=0}^{\log N-1} \left(\omega^{2^{\log N-i}} \right)^{X \cdot y_i} \quad (15)$$

$$= \omega^X \cdot \sum_{y \in \{0,1\}^{\log N}} \prod_{i=0}^{\log N-1} ((1 - u_i)(1 - y_i) + u_i y_i) \cdot \omega_{2^{i+1}}^{X \cdot y_i} \quad (16)$$

$$= \omega^X \cdot \prod_{i=0}^{\log N-1} (1 - u_i + u_i \cdot \omega_{2^{i+1}}^X) \quad (17)$$

$$= \omega^{\sum_{j=0}^{\log N-1} 2^j \cdot x_j} \cdot \prod_{i=0}^{\log N-1} (1 - u_i + u_i \cdot \omega_{2^{i+1}}^X) \quad (18)$$

$$= \prod_{i=0}^{\log N-1} (1 - u_i + u_i \cdot \omega_{2^{i+1}}^X) \cdot \omega^{2^i \cdot x_i} \quad (19)$$

F DEFINITION OF HASTEBOOTS

We say that HasteBoots is a SNARG for the indexed relation $\mathcal{R}_{\text{NAND}}$ if the following properties hold.

- **Completeness.** For any $(i, \mathbb{x}, \mathbb{w}) \in \mathcal{R}_{\text{NAND}}$, $\text{pp} \leftarrow \text{PPGen}(1^\lambda)$, $(\text{ipk}, \text{ivk}) \leftarrow \text{KeyGen}(\mathbb{i}, \text{pp})$, and $\pi \leftarrow \text{Prove}(\mathbb{x}, \text{ipk})$, we have $\Pr[\text{Verify}(\mathbb{x}, \text{ivk}, \pi) = 1] = 1$.
- **Soundness.** For any PPT adversary $\mathcal{A}$, the following probability is negligible in λ :

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{PPGen}(1^\lambda) \\ (\text{ipk}, \text{ivk}) \leftarrow \text{KeyGen}(\mathbb{i}) \\ (\mathbb{x}, \pi^*) \leftarrow \mathcal{A}(1^\lambda, \text{pp}, \text{st}) \\ \text{Verify}(\mathbb{x}, \text{ivk}, \pi^*, \text{pp}) = 1 \end{array} \right].$$

G EVALUATION

G.1 Asymptotic Complexity

We analyze the asymptotic complexity for all atomic and FHE operations in Table 4. In our implementations for a single NAND operation using the parameter settings described above, we compute the following $L_1 = 5, L_2 = 3, M_1 \approx 4L_1n + 2n = 22528, M_2 \approx 2L_1n = 10240, M_3 = n + 1 = 1025$. A single NAND operation involves approximately $n(2L_1 + 2) = 12288$ NTT instances. The complexity of the Brakedown PCS, as described in [41], leads to $O(N)$ proving time, and $O(\sqrt{N})$ verification time and proof size, where N denotes the size of the polynomial. In comparison, the BaseFold PCS [68] offers $O(N \log N)$ proving time and $O(\log N)$ verification time and proof size.

G.2 Parameter Comparison to [61]

The parameters in our work are not exactly the same as those in [61], but they are quite similar, as many parameters in FHE need to be tweaked and optimized to ensure both correctness and security. In [61], they choose a parameter setting with $n = 728$, with Q being a 64-bit prime while their implementation also uses a 128-bit extension field of size Q^2 . It is important to note that the circuit size grows with n , which means that we are effectively proving the correctness of a larger circuit compared to the one in their implementation.